

Concurrency interview questions

04/24/2026 16:39:05

Give questions on concurrency in java 8 year experience

View: Check available skills Here are Java Concurrency Interview Questions for 8 years of experience,...

Now answer all these questions

I'll answer all 35 questions comprehensively. Let me create a well-structured document for this. View: Read...

Don't create documents write the answer here only

Sure! Here are all 35 answers: 🔴 Core Concepts & Fundamentals 1. What is the Java Memory Model (JMM)...

Write vs comparison question answer on this context on this chat

Here are the key vs comparison Q&As from the Java Concurrency topic: 1. synchronized vs volatile vs Atomi...

Jpmc concurrency questions and answers for 8 years of experience

Here are JPMC-style Java Concurrency Interview Questions & Answers for 8 years of experience. JPMC...

What is bucket in hashmap

What is a Bucket in HashMap? Simple Definition A bucket is just an index slot in an internal array inside a...

What is this things ReentrantLock, Semaphore, CountdownLatch, CyclicBarrier

Great question! Let me explain each one simply with real-world analogies and code. 1. ReentrantLock 🗝️ ...

MBB Labs all concurrency questions ? concurrency scenario based questions

Here are MBB Labs (McKinsey, Bain, BCG style tech arm) Concurrency Scenario-Based Questions & Answers...

anything missed in multithreading or concurrency

Here are all the topics missed — covering every corner of Java Multithreading & Concurrency: 🔴 1. Thread...

java8 addition to multithreading or concurrency added here ?

Here are all Java 8 additions to Multithreading & Concurrency specifically: 🔴 1. CompletableFuture — Bigge...

all multithreading and concurrency question answwr covered here or pending

Let me do a complete audit of everything covered and what's still pending: ✅ COVERED TOPICS # Topic...

yes any scenario based questions missed add it here also

Here are all 14 pending topics + missed scenario-based questions: 🔴 PENDING 1: Biased / Thin / Fat Lock...

3/7/2026, 5:51:21 AM

View: Check available skills

Here are **Java Concurrency Interview Questions** for 8 years of experience, covering advanced topics:

Core Concepts & Fundamentals

1. **What is the Java Memory Model (JMM)? Explain happens-before relationships.**
2. Explain the difference between `synchronized`, `volatile`, and `Atomic` variables. When would you choose each?
3. What are the problems with using `synchronized` excessively? How do you minimize lock contention?
4. What is a **race condition** vs a **data race**? Can you have one without the other?
5. Explain **false sharing** in CPU caches and how to prevent it in Java (e.g., padding, `@Contended`).

Locks & Synchronization

6. Compare `ReentrantLock` vs `synchronized`. When is `ReentrantLock` preferable?
7. What is a **deadlock**, **livelock**, and **starvation**? How do you detect and prevent each?
8. Explain `ReadWriteLock` and `StampedLock`. When does `StampedLock` outperform `ReadWriteLock`?
9. What is a **spin lock**? When is busy-waiting acceptable in Java?
10. How does `LockSupport.park()` / `unpark()` work? How is it different from `Object.wait()`?

Java Concurrency Utilities (`java.util.concurrent`)

11. Explain the internals of `ConcurrentHashMap`. How does it achieve lock-free reads in Java 8?
12. What is the difference between `CopyOnWriteArrayList` and `Collections.synchronizedList()`? When should each be used?
13. Explain `CountDownLatch` vs `CyclicBarrier` vs `Phaser`. Give a real-world use case for each.

14. What is a `Semaphore`? How would you implement a connection pool using it?
 15. How does `BlockingQueue` work? Compare `ArrayBlockingQueue`, `LinkedBlockingQueue`, and `SynchronousQueue`.
-

● Thread Pools & ExecutorService

16. Explain the `ThreadPoolExecutor` parameters: `corePoolSize`, `maximumPoolSize`, `keepAliveTime`, and the `RejectedExecutionHandler`. What happens when the queue is full?
 17. What are the trade-offs between fixed, cached, and work-stealing thread pools?
 18. How would you implement a **rate limiter** using Java concurrency primitives?
 19. How do you gracefully shut down a thread pool? What's the difference between `shutdown()` and `shutdownNow()`?
 20. What is **thread starvation** in a thread pool and how do you diagnose it?
-

● CompletableFuture & Async Programming

21. Explain `CompletableFuture` vs `Future`. What limitations of `Future` does it address?
 22. What is the difference between `thenApply()`, `thenCompose()`, and `thenCombine()`?
 23. How does exception handling work in `CompletableFuture`? Explain `exceptionally()` vs `handle()` vs `whenComplete()`.
 24. How do you run multiple async tasks in parallel and wait for all/any to complete?
 25. What thread pool does `CompletableFuture.supplyAsync()` use by default? What are the risks of using the common pool?
-

● Advanced & Design Questions

26. How would you design a **producer-consumer** system in Java that handles back-pressure?
27. Explain the **ABA problem** in lock-free programming. How does `AtomicStampedReference` solve it?
28. What is **compare-and-swap (CAS)**? How do Java's `Atomic*` classes use it under the hood?
29. How would you implement a **thread-safe singleton** in Java? Discuss double-checked locking and its pitfalls.
30. You're seeing intermittent `NullPointerException`s in a multi-threaded context. Walk me through how you would diagnose it.

Debugging & Production Scenarios

31. How do you detect a deadlock in a running JVM? What tools do you use?
32. What is a **thread dump** and how do you analyze it? What would you look for?
33. How do you identify **memory visibility** bugs that only occur under high load?
34. What profiling tools have you used to diagnose concurrency issues (VisualVM, JFR, async-profiler)?
35. Describe a real concurrency bug you've debugged in production. What was the root cause and fix?

Tip for interviews at this level: Expect follow-ups on *why* and *internals*. Be ready to sketch out the `ConcurrentHashMap` segment structure or explain how the JVM maps Java threads to OS threads.

3/7/2026, 5:51:47 AM



Now answer all these questions

3/7/2026, 5:56:04 AM

I'll answer all 35 questions comprehensively. Let me create a well-structured document for this.

View: Read docx skill

Now I'll create a comprehensive Word document with all 35 answers.

Bash Tool: Install docx npm package

3/7/2026, 5:56:11 AM

3/7/2026, 5:57:52 AM

Sure! Here are all 35 answers:

Core Concepts & Fundamentals

1. What is the Java Memory Model (JMM)? Explain happens-before relationships.

The JMM defines how threads interact through memory. Each thread has its own local cache, and the JMM governs when writes by one thread become visible to another.

Happens-before is a guarantee that if action A happens-before action B, then A's memory writes are visible to B. Key happens-before rules:

- A `synchronized` block's unlock happens-before every subsequent lock on the same monitor
- A `volatile` write happens-before every subsequent read of that variable
- `Thread.start()` happens-before any action in the started thread
- `Thread.join()` — all actions in the joined thread happen-before `join()` returns

Without happens-before, the JVM/CPU can reorder instructions freely, causing visibility bugs.

2. Difference between `synchronized`, `volatile`, and `Atomic` variables.

Feature	<code>synchronized</code>	<code>volatile</code>	<code>Atomic</code>
Mutual exclusion	✓ Yes	✗ No	✗ No
Visibility	✓ Yes	✓ Yes	✓ Yes
Compound ops (check-then-act)	✓ Yes	✗ No	✓ Yes (CAS)
Performance	Slower (lock)	Fast	Fast (CAS)

- Use `volatile` for a single flag/boolean that one thread writes and others read.
- Use `Atomic*` for counters or single-variable compound operations.
- Use `synchronized` when you need to protect a block involving multiple variables.

3. Problems with excessive `synchronized` and minimizing lock contention.

Problems:

- **Thread starvation** — lower priority threads never get the lock
- **Deadlock** — two threads waiting on each other's locks
- **Reduced throughput** — threads queue up, CPU sits idle
- **Context switching overhead** — expensive OS-level switches

Solutions:

- Narrow the synchronized block to the minimum critical section
 - Use `ReadWriteLock` when reads vastly outnumber writes
 - Use `ConcurrentHashMap` instead of `synchronizedMap`
 - Use lock striping (as `ConcurrentHashMap` does internally)
 - Prefer `Atomic*` or `volatile` when applicable
-

4. Race condition vs data race — can you have one without the other?

- **Data race:** Two threads access the same variable concurrently, at least one writes, and there's no synchronization. This is a JMM violation.
- **Race condition:** A bug where correctness depends on the timing/ordering of threads — even with synchronization.

Yes, you can have one without the other:

- **Data race without race condition:** Two threads write the same value simultaneously — technically a data race, but no observable bug.
 - **Race condition without data race:** A properly synchronized check-then-act (e.g., `if (!map.containsKey(k)) map.put(k, v)` with each call synchronized separately) — no data race, but the compound operation is still racey.
-

5. False sharing and how to prevent it.

CPUs cache memory in **cache lines** (~64 bytes). If two threads write to different variables that happen to sit on the same cache line, each write invalidates the other core's cache — this is false sharing. It causes massive performance degradation with no logical sharing.

Prevention:

- **Padding:** Add dummy fields to push variables to separate cache lines
- **@Contended annotation** (JDK 8+): JVM automatically pads the field. Requires **-XX:-RestrictContended** JVM flag
- **Example:** **LongAdder** uses **@Contended** on its **Cell** class internally

```
@sun.misc.Contended
static final class Cell {
    volatile long value;
}
```

🔴 Locks & Synchronization

6. **ReentrantLock** vs **synchronized**.

Feature	synchronized	ReentrantLock
Fairness policy	❌ No	✅ Optional (new ReentrantLock(true))
Try to acquire (non-blocking)	❌ No	✅ tryLock()
Timed lock attempt	❌ No	✅ tryLock(time, unit)
Interruptible lock wait	❌ No	✅ lockInterruptibly()
Multiple conditions	❌ No	✅ newCondition()
Automatic release	✅ Yes	❌ Must use try-finally

Use **ReentrantLock** when you need timeout, fairness, multiple condition queues, or interruptibility. Prefer **synchronized** for simplicity.

7. Deadlock, livelock, and starvation.

- **Deadlock:** Thread A holds Lock 1 and waits for Lock 2; Thread B holds Lock 2 and waits for Lock 1. Both block forever.
 - *Prevention:* Always acquire locks in the same global order; use **tryLock()** with timeout.
- **Livelock:** Threads keep changing state in response to each other but make no progress (like two people in a hallway dodging each other).
 - *Prevention:* Introduce randomness or back-off in retry logic.

- **Starvation:** A thread is perpetually denied CPU time because other threads always get priority.
 - *Prevention:* Use fair locks (`new ReentrantLock(true)`), avoid priority inversion.
-

8. `ReadWriteLock` vs `StampedLock`.

`ReadWriteLock` allows concurrent reads but exclusive writes. Multiple readers can proceed simultaneously; writers block all.

`StampedLock` (Java 8) adds **optimistic reading** — a reader doesn't acquire a lock at all; it reads and then validates if no write occurred. If validation fails, it falls back to a real read lock.

`StampedLock` outperforms `ReadWriteLock` when:

- Reads are very frequent and writes are rare
- You can tolerate retry logic for optimistic reads

Limitation: `StampedLock` is **not reentrant**, so the same thread cannot acquire it twice.

9. Spin lock and when busy-waiting is acceptable.

A spin lock loops continuously checking if a lock is free instead of blocking the thread. In Java, you can simulate with:

```
while (!lock.compareAndSet(false, true)) { /* spin */ }
```

Busy-waiting is acceptable when:

- The wait time is expected to be **very short** (nanoseconds to microseconds)
- You're running on **multi-core** hardware (single-core spinning starves others)
- Context switch cost exceeds the expected wait time
- In real-time or low-latency systems (HFT, game engines)

For general application code, busy-waiting is almost always a bad idea.

10. `LockSupport.park()` vs `Object.wait()`.

Feature	<code>LockSupport.park()</code>	<code>Object.wait()</code>
Requires monitor	✗ No	✓ Must be in <code>synchronized</code> block
Spurious wakeups	Must handle	Must handle
Permit-based	✓ Yes (pre-permit possible)	✗ No
Unpark before park	✓ Works correctly	✗ <code>notify()</code> before <code>wait()</code> is lost

`LockSupport` is the foundation of `AbstractQueuedSynchronizer` (AQS), which powers `ReentrantLock`, `Semaphore`, `CountDownLatch`, etc. The permit model means `unpark()` called before `park()` will let the thread continue immediately — unlike `notify()` which is lost if called before `wait()`.

Java Concurrency Utilities

11. Internals of `ConcurrentHashMap` in Java 8.

Java 8 completely rewrote `ConcurrentHashMap`:

- **No more segment locks** (the Java 6 approach). Instead, it locks only at the **individual bucket (bin) level** using `synchronized` on the first node.
- **Reads are lock-free** — the table array and nodes use `volatile`, so readers never block.
- When a bin exceeds 8 entries, it converts from a linked list to a **red-black tree** (TreeBin) for $O(\log n)$ lookup.
- **CAS** is used for inserting the first node into an empty bin — no locking needed.
- `size()` uses a `baseCount` + `CounterCell[]` approach (similar to `LongAdder`) to reduce contention on updates.

12. `CopyOnWriteArrayList` vs `Collections.synchronizedList()`.

Feature	<code>CopyOnWriteArrayList</code>	<code>synchronizedList</code>
Read locking	✗ None	✓ Every read locks
Write cost	High (full array copy)	Low
Iterator safety	✓ Snapshot, no <code>ConcurrentModificationException</code>	✗ Must manually sync
Best for	Read-heavy, rare writes	Balanced read/write

Use `CopyOnWriteArrayList` for **event listener lists** or **observer patterns** where reads vastly dominate and the list is small. Never use it for large lists with frequent writes — the $O(n)$ copy on every write is too expensive.

13. `CountDownLatch` vs `CyclicBarrier` vs `Phaser` .

- `CountDownLatch` : One-time gate. One or more threads wait for a count to reach zero. Cannot be reset.
 - *Use case*: Wait for all services to initialize before starting the app.
- `CyclicBarrier` : All threads wait at a barrier until all arrive, then proceed together. Can be reused (cyclic).
 - *Use case*: Parallel computation where each round needs all threads to finish before the next round starts.
- `Phaser` : Like `CyclicBarrier` but with dynamic party registration/deregistration and multiple phases.
 - *Use case*: Multi-phase parallel pipelines where threads may join/leave between phases.

14. `Semaphore` and connection pool implementation.

A `Semaphore` maintains a count of permits. `acquire()` decrements (blocks if 0), `release()` increments.

```
public class ConnectionPool {
    private final Semaphore semaphore;
    private final BlockingQueue<Connection> pool;

    public ConnectionPool(int size) {
        semaphore = new Semaphore(size, true); // fair
        pool = new ArrayBlockingQueue<>(size);
        for (int i = 0; i < size; i++) pool.offer(createConnection());
    }

    public Connection acquire() throws InterruptedException {
        semaphore.acquire();
        return pool.poll();
    }

    public void release(Connection conn) {
        pool.offer(conn);
        semaphore.release();
    }
}
```

15. `BlockingQueue` implementations compared.

- `ArrayBlockingQueue`: Bounded, backed by array. Single lock for both put and take. Fair ordering optional. Good when you want to bound memory.
- `LinkedBlockingQueue`: Optionally bounded (default `Integer.MAX_VALUE`). Separate locks for put and take — higher throughput under contention. Risk of unbounded memory growth if not capped.
- `SynchronousQueue`: Zero capacity. Every `put()` blocks until a thread calls `take()` — a direct handoff. Used internally by `Executors.newCachedThreadPool()`. No buffering at all.
- `PriorityBlockingQueue`: Unbounded, elements ordered by priority. Used in task schedulers.
- `LinkedTransferQueue`: Like `SynchronousQueue` + `LinkedBlockingQueue`. Producers can optionally wait for a consumer (`transfer()`) or not (`put()`).

Thread Pools & `ExecutorService`

16. `ThreadPoolExecutor` parameters and queue-full behavior.

```
ThreadPoolExecutor(corePoolSize, maximumPoolSize, keepAliveTime, unit, workQueue, handler)
```

- `corePoolSize`: Threads always kept alive (even idle)
- `maximumPoolSize`: Max threads ever created
- `keepAliveTime`: How long excess (above core) threads stay idle before termination
- `workQueue`: Holds tasks when all core threads are busy

Flow when a task is submitted:

1. If fewer than `corePoolSize` threads → create new thread
2. If at or above core → try to queue the task
3. If queue is full and below max → create new thread
4. If queue full and at max → invoke `RejectedExecutionHandler`

Rejection policies: `AbortPolicy` (throws), `CallerRunsPolicy` (runs in caller's thread — provides back-pressure), `DiscardPolicy` (silently drops), `DiscardOldestPolicy` (drops oldest queued task).

17. Fixed vs cached vs work-stealing thread pools.

- `newFixedThreadPool(n)`: Fixed number of threads, unbounded `LinkedListBlockingQueue`. Risk: queue grows unboundedly under sustained load.
- `newCachedThreadPool()`: Unlimited threads, `SynchronousQueue`. Creates new threads on demand, reuses idle ones. Risk: thread explosion under burst load.
- `newWorkStealingPool()` (Java 8): Uses `ForkJoinPool`. Each thread has its own deque. Idle threads steal tasks from the tail of busy threads' deques. Best for **CPU-bound, recursive/divide-and-conquer tasks** (e.g., parallel streams, recursive algorithms).

For most server workloads: fixed pool with bounded queue + `CallerRunsPolicy` is the safest choice.

18. Rate limiter using Java concurrency primitives.

```
public class RateLimiter {
    private final Semaphore semaphore;
    private final ScheduledExecutorService scheduler;

    public RateLimiter(int permitsPerSecond) {
        this.semaphore = new Semaphore(permitsPerSecond);
        this.scheduler = Executors.newScheduledThreadPool(1);
        scheduler.scheduleAtFixedRate(
            () -> semaphore.release(
                permitsPerSecond - semaphore.availablePermits()
            ),
            1, 1, TimeUnit.SECONDS
        );
    }

    public void acquire() throws InterruptedException {
        semaphore.acquire();
    }
}
```

For production, Google Guava's `RateLimiter` uses a token bucket algorithm with smooth bursty and warm-up modes.

19. Graceful thread pool shutdown — `shutdown()` vs `shutdownNow()`.

- `shutdown()`: Initiates orderly shutdown. No new tasks accepted, but **queued and running tasks complete normally**. Non-blocking — you must call `awaitTermination()` to wait.
- `shutdownNow()`: Attempts to stop all running tasks by calling `Thread.interrupt()` on them. Returns the list of queued tasks that were never started. Tasks that don't respond to interruption won't stop.

Graceful pattern:

```
executor.shutdown();
if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
    executor.shutdownNow();
    if (!executor.awaitTermination(10, TimeUnit.SECONDS)) {
        log.error("Executor did not terminate");
    }
}
```

20. Thread starvation in a thread pool and how to diagnose it.

Starvation in a pool happens when long-running tasks occupy all threads and shorter tasks can't execute — or when a task submits a subtask to the same pool and waits for it (a **deadlock by pool exhaustion**).

Diagnosis:

- **Thread dumps** (`jstack`, VisualVM) — look for threads in WAITING state all blocked on pool queue
- **JMX / MBeans** — monitor `ThreadPoolExecutor` via JMX: active count, queue size, completed tasks
- **Metrics** — track queue depth over time; sustained growth indicates starvation
- Look for tasks that call `Future.get()` inside the pool (a classic self-deadlock pattern)

Fix: use a separate pool for subtasks, or use `ForkJoinPool` which handles this natively with work-stealing.

CompletableFuture & Async Programming

21. `CompletableFuture` vs `Future`.

`Future` limitations:

- `get()` is **blocking** — no way to attach a callback
- Cannot chain operations
- Cannot combine multiple futures

- No exception handling hooks
- Cannot be manually completed

`CompletableFuture` solves all of these:

- Non-blocking callbacks: `thenApply`, `thenAccept`, `thenRun`
- Chaining and composition: `thenCompose`, `thenCombine`
- Exception handling: `exceptionally`, `handle`
- Manual completion: `complete(value)`, `completeExceptionally(ex)`
- Combining: `allOf`, `anyOf`

22. `thenApply()` vs `thenCompose()` vs `thenCombine()`.

- `thenApply(fn)`: Transforms the result synchronously. `fn` takes `T`, returns `U`. Like `map()` on a stream.

```
CompletableFuture<String> f = future.thenApply(n -> "Hello " + n);
```

- `thenCompose(fn)`: Chains two async operations. `fn` takes `T`, returns `CompletableFuture<U>`. Like `flatMap()`. Use when the next step is also async.

```
CompletableFuture<Order> f = getUserId().thenCompose(id -> fetchOrder(id));
```

- `thenCombine(other, fn)`: Waits for **two independent futures** to complete, then combines their results.

```
CompletableFuture<String> f = future1.thenCombine(future2, (a, b) -> a + b);
```

23. Exception handling in `CompletableFuture`.

- `exceptionally(fn)`: Called only on failure. Provides a fallback value. The happy path is unaffected.

```
future.exceptionally(ex -> "default");
```

- `handle(fn)`: Called on both success and failure. Receives `(result, exception)` — one will be null.

```
future.handle((result, ex) -> ex != null ? "error" : result);
```

- `whenComplete(fn)`: Like `handle` but **doesn't transform** the result — used for side effects (logging). The original result/exception propagates.

```
future.whenComplete((result, ex) -> log.info("Done: {}", result));
```

Key point: exceptions in `CompletableFuture` are wrapped in `CompletionException`. Use `ex.getCause()` to get the original.

24. Running multiple async tasks in parallel.

```
// Wait for ALL to complete
CompletableFuture<Void> all = CompletableFuture.allOf(f1, f2, f3);
all.join(); // results in f1.join(), f2.join(), f3.join()

// Wait for ANY to complete
CompletableFuture<Object> any = CompletableFuture.anyOf(f1, f2, f3);
Object first = any.join();

// Collect all results into a list
List<CompletableFuture<String>> futures = List.of(f1, f2, f3);
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0]))
    .thenApply(v ->
futures.stream().map(CompletableFuture::join).collect(toList()));
```

25. Default thread pool for `supplyAsync()` and risks.

By default, `CompletableFuture.supplyAsync()` uses the `ForkJoinPool.commonPool()` — a shared, JVM-wide pool with threads = `Runtime.availableProcessors() - 1`.

Risks:

- **Shared pool starvation**: If your async tasks are I/O-bound and block, they occupy common pool threads, starving other users (parallel streams also use the common pool)
- **Unpredictable sizing**: The pool size is fixed at JVM startup
- **Long-blocking tasks** bring the whole JVM's parallelism down

Best practice: always supply a dedicated `Executor` for I/O-bound tasks:

```
CompletableFuture.supplyAsync(() -> fetchFromDB(), myIOExecutor);
```

🔴 Advanced & Design Questions

26. Producer-consumer with back-pressure.

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(100); // bounded = back-  
pressure  
  
// Producer  
executor.submit(() -> {  
    while (!done) {  
        Task t = generateTask();  
        queue.put(t); // blocks when full – natural back-pressure  
    }  
});  
  
// Consumer  
executor.submit(() -> {  
    while (!done || !queue.isEmpty()) {  
        Task t = queue.poll(1, TimeUnit.SECONDS);  
        if (t != null) process(t);  
    }  
});
```

For reactive back-pressure at scale, use **Project Reactor** (**Flux** with **onBackpressureBuffer** / **DROP** / **ERROR**) or **RxJava**. The key principle is: the bounded queue causes the producer to slow down naturally when consumers can't keep up.

27. ABA problem and **AtomicStampedReference**.

ABA problem: Thread reads value A, another thread changes A → B → A. When the first thread does CAS(A, newValue), it succeeds even though the value was changed and changed back — the logic may be wrong (e.g., in a lock-free stack, a node was removed and re-inserted).

AtomicStampedReference pairs the value with an integer **stamp** (version counter). CAS checks **both** the value and the stamp:

```
AtomicStampedReference<Node> ref = new AtomicStampedReference<>(node, 0);
int[] stamp = new int[1];
Node current = ref.get(stamp);
ref.compareAndSet(current, newNode, stamp[0], stamp[0] + 1);
```

Now even if value goes $A \rightarrow B \rightarrow A$, the stamp will be different ($0 \rightarrow 1 \rightarrow 2$), so the stale CAS fails correctly.

28. Compare-and-swap (CAS) and `Atomic*` internals.

CAS is a CPU instruction (`CMPXCHG` on x86) that atomically does:

```
if (memory == expected) { memory = newValue; return true; }
else return false;
```

Java's `Atomic*` classes (e.g., `AtomicInteger`) use `sun.misc.Unsafe.compareAndSwapInt()` which maps directly to this CPU instruction — no OS lock, no context switch.

`AtomicInteger.incrementAndGet()` internally does:

```
do {
    current = get();
} while (!compareAndSet(current, current + 1));
```

This is a **spin loop** on CAS. Under low contention it's extremely fast. Under high contention, many threads retry repeatedly — this is why `LongAdder` was introduced in Java 8, spreading the contention across multiple cells.

29. Thread-safe singleton — double-checked locking pitfalls.

Naive double-checked locking is broken without `volatile`:

```
// BROKEN without volatile
private static Singleton instance;
public static Singleton getInstance() {
    if (instance == null) { // check 1
        synchronized (Singleton.class) {
            if (instance == null) { // check 2
                instance = new Singleton(); // partially constructed object may be
                visible!
            }
        }
    }
}
```

```

        }
    }
}
return instance;
}

```

The fix is `private static volatile Singleton instance;`. Without `volatile`, the JVM can reorder the write to `instance` before the constructor finishes.

Best approaches:

- `volatile` + **double-checked locking** (above, fixed)
- **Initialization-on-demand holder** (preferred — lazy, thread-safe, no volatile):

```

private static class Holder {
    static final Singleton INSTANCE = new Singleton();
}

public static Singleton getInstance() { return Holder.INSTANCE; }

```

- `enum` **singleton**: Simplest, serialization-safe, reflection-safe.

30. Diagnosing intermittent `NullPointerException` in multi-threaded context.

Step-by-step approach:

1. **Identify the field**: Which variable is null? Is it shared across threads?
2. **Check visibility**: Is it declared `volatile` or accessed under a lock? Without this, one thread may see a stale null even after another has initialized it.
3. **Check publication**: Is the object safely published? Writing to a shared reference without synchronization can expose partially constructed objects.
4. **Check for race on lazy init**: `if (x == null) x = new X()` without synchronization is a classic race.
5. **Add logging/assertions**: Log thread name + variable state on access.
6. **Use thread sanitizers or tools**: Java Race Detector doesn't exist natively, but **async-profiler** and **JFR** can help trace concurrent access patterns.
7. **Add synchronization or use** `AtomicReference` to fix once the race is confirmed.

Debugging & Production Scenarios

31. Detecting deadlock in a running JVM.

- `jstack <pid>`: Prints thread dump. JVM automatically detects deadlocks and prints:

```
Found one Java-level deadlock:
Thread-1 waiting for Thread-2's lock
Thread-2 waiting for Thread-1's lock
```

- `jconsole` / `jvisualvm`: GUI tools with a "Detect Deadlock" button under the Threads tab.
- `ThreadMXBean`:

```
ThreadMXBean bean = ManagementFactory.getThreadMXBean();
long[] ids = bean.findDeadlockedThreads();
```

- **JFR (Java Flight Recorder)**: Continuous low-overhead profiling that captures lock contention events.

32. Thread dump analysis — what to look for.

A thread dump is a snapshot of all thread states. Key things to look for:

- **BLOCKED**: Thread waiting to acquire a monitor held by another — look for the "waiting to lock" address and find who owns it.
- **WAITING / TIMED_WAITING**: Thread called `wait()`, `park()`, `sleep()`. Check if expected or stuck.
- **Deadlock section**: JVM prints this automatically if detected.
- **All threads in BLOCKED on same lock**: Lock contention bottleneck.
- **Thread pool threads all WAITING**: Pool is idle — maybe tasks aren't being submitted.
- **Thread pool threads all BLOCKED**: Possible starvation or deadlock.
- Take **3 dumps 10 seconds apart** — threads stuck in the same state across all dumps are truly blocked.

33. Identifying memory visibility bugs under high load.

Memory visibility bugs are the hardest to find because they're non-deterministic and often disappear under a debugger (which adds synchronization implicitly).

Approach:

- **Code review for missing** `volatile` / `synchronized`: Look for shared fields written by one thread and read by another without coordination.

- **Use `-server` flag in testing:** Server JIT is more aggressive at reordering, making visibility bugs more likely to manifest.
 - **Stress testing with many threads:** Visibility bugs appear under load when CPU caches diverge.
 - **Java Flight Recorder:** Monitor for unexpected stale reads using custom events.
 - **Static analysis tools:** SpotBugs (FindBugs successor) flags missing volatile on shared fields.
 - **`@GuardedBy` annotations:** Document intended locking and use tools to verify.
-

34. Profiling tools for concurrency issues.

- **`jstack`:** Quick thread dump for deadlock/starvation analysis. No overhead.
 - **`jconsole` / `VisualVM`:** GUI tools showing thread states, heap, CPU. Good for real-time monitoring.
 - **JFR (Java Flight Recorder) + JMC (Java Mission Control):** Low-overhead (~1%) continuous profiling. Captures lock contention, thread sleep, GC events. Best for production.
 - **async-profiler:** Samples CPU and wall-clock time without safepoint bias. Excellent for finding where threads actually spend time (including blocked time). Produces flame graphs.
 - **`ThreadMXBean`:** Programmatic access to thread stats, CPU time per thread, contended monitor info.
 - **Arthas (Alibaba):** Dynamic Java diagnostic tool — can trace method calls, watch thread states in live production without restart.
-

35. Real concurrency bug in production — example.

Scenario: A high-traffic service used a lazy-initialized cache:

```
private Map<String, Config> cache;

public Config getConfig(String key) {
    if (cache == null) {
        cache = loadFromDB(); // expensive
    }
    return cache.get(key);
}
```

Symptoms: Occasional `NullPointerException` and DB being hit far more than expected — especially at startup under load.

Root cause: Two races:

1. `cache` was not `volatile`, so some threads saw it as null even after initialization.
2. Multiple threads passed the null check simultaneously, each calling `loadFromDB()` — wasting resources and potentially causing DB overload.

Fix:

```
private volatile Map<String, Config> cache;

public Config getConfig(String key) {
    if (cache == null) {
        synchronized (this) {
            if (cache == null) {
                cache = Collections.unmodifiableMap(loadFromDB());
            }
        }
    }
    return cache.get(key);
}
```

Or better — use `AtomicReference` with `compareAndSet`, or simply eagerly initialize in a `@PostConstruct` method.

Lesson: Any shared mutable reference requires either `volatile`, `synchronized`, or an atomic wrapper. "It works in testing" means nothing for visibility bugs — they emerge under production load.

3/7/2026, 6:00:14 AM

3/8/2026, 1:27:53 PM

Here are the key **vs comparison Q&As** from the Java Concurrency topic:

1. **synchronized** vs **volatile** vs **Atomic**

	synchronized	volatile	Atomic
Mutual Exclusion	✓	✗	✗
Visibility	✓	✓	✓
Compound Operations	✓	✗	✓ (CAS)
Blocking	✓ Blocks	✗	✗
Performance	Slowest	Fastest	Fast

When to use:

- **volatile** → single flag, one writer, many readers
- **Atomic** → counters, single-variable CAS operations
- **synchronized** → multi-variable critical sections

2. **ReentrantLock** vs **synchronized**

	ReentrantLock	synchronized
Fairness	✓ Optional	✗ No
tryLock()	✓ Yes	✗ No
Timed lock	✓ Yes	✗ No
Interruptible	✓ Yes	✗ No
Multiple conditions	✓ newCondition()	✗ One only
Auto release	✗ Manual try-finally	✓ Auto
Simplicity	✗ Verbose	✓ Simple

When to use `ReentrantLock`: timeout, fairness, multiple condition queues, interruptible waits. Otherwise prefer `synchronized`.

3. `ReadWriteLock` vs `StampedLock`

	<code>ReadWriteLock</code>	<code>StampedLock</code>
Concurrent reads	✓	✓
Optimistic read	✗	✓ (no lock at all)
Reentrancy	✓	✗ Not reentrant
Complexity	Low	High
Performance (read-heavy)	Good	Better

Key difference: `StampedLock` lets you read without acquiring any lock, then validate. If a write happened, retry with a real read lock. Wins big when writes are rare.

4. `CountDownLatch` vs `CyclicBarrier` vs `Phaser`

	<code>CountDownLatch</code>	<code>CyclicBarrier</code>	<code>Phaser</code>
Reusable	✗ One-time	✓ Yes	✓ Yes
Dynamic parties	✗	✗	✓
Who counts down	Any thread	Participating threads	Participating threads
Multiple phases	✗	✗	✓
Barrier action	✗	✓ Optional runnable	✓

- `CountDownLatch` → wait for N events to happen (e.g., services to boot)
- `CyclicBarrier` → all threads meet at a point, repeat each round
- `Phaser` → multi-phase pipelines with threads joining/leaving dynamically

5. `shutdown()` vs `shutdownNow()`

	<code>shutdown()</code>	<code>shutdownNow()</code>
Accepts new tasks	✗	✗
Running tasks	✓ Completes them	✗ Interrupts them
Queued tasks	✓ Executes them	✗ Returns them as list
Blocking	✗ (use <code>awaitTermination()</code>)	✗
Use case	Graceful shutdown	Emergency/forced stop

6. `Future` vs `CompletableFuture`

	<code>Future</code>	<code>CompletableFuture</code>
Non-blocking callback	✗	✓
Chaining	✗	✓
Combine multiple	✗	✓ <code>allOf</code> / <code>anyOf</code>
Exception handling	✗ Only on <code>get()</code>	✓ <code>exceptionally</code> , <code>handle</code>
Manual completion	✗	✓ <code>complete()</code>
Async pipeline	✗	✓

7. `thenApply()` vs `thenCompose()` vs `thenCombine()`

	<code>thenApply</code>	<code>thenCompose</code>	<code>thenCombine</code>
Analogy	<code>map</code>	<code>flatMap</code>	<code>zip</code>
Input	T	T	T + U (two futures)
Output	<code>CF<U></code>	<code>CF<U></code> (unwrapped)	<code>CF<V></code>
Next step async?	✗	✓	✓ (independent)
Use case	Transform result	Chain async calls	Merge two parallel results

8. `exceptionally()` vs `handle()` vs `whenComplete()`

	exceptionally	handle	whenComplete
Called on success	✗	✓	✓
Called on failure	✓	✓	✓
Can transform result	✓ (fallback)	✓	✗ (side-effect only)
Returns new CF	✓	✓	✗ (same result propagates)
Use case	Fallback value	Recovery/transform	Logging/cleanup

9. **ArrayBlockingQueue** vs **LinkedBlockingQueue** vs **SynchronousQueue**

	ArrayBlockingQueue	LinkedBlockingQueue	SynchronousQueue
Capacity	Bounded (required)	Optional bound	Zero (no buffer)
Backing structure	Array	Linked nodes	None
Lock strategy	Single lock	Separate put/take locks	CAS
Throughput	Moderate	Higher	Highest (direct handoff)
Memory	Fixed	Dynamic	Minimal
Use case	Bounded work queue	General producer-consumer	newCachedThreadPool

10. **CopyOnWriteArrayList** vs **synchronizedList**

	CopyOnWriteArrayList	synchronizedList
Read locking	✗ None	✓ Every read
Write cost	O(n) — full copy	O(1) — just locks
Iterator safety	✓ Snapshot, no CME	✗ Must sync externally
Best for	Rare writes, many reads	Balanced reads/writes
Memory	Higher (copies)	Lower

11. Fixed vs Cached vs Work-Stealing Thread Pool

	<code>newFixedThreadPool</code>	<code>newCachedThreadPool</code>	<code>newWorkStealingPool</code>
Thread count	Fixed	Unlimited	CPU cores
Queue	Unbounded linked	<code>SynchronousQueue</code>	Per-thread dequeues
Risk	Queue overflow	Thread explosion	Task ordering unpredictable
Best for	Stable server load	Short bursts	CPU-bound recursive tasks
Underlying	<code>ThreadPoolExecutor</code>	<code>ThreadPoolExecutor</code>	<code>ForkJoinPool</code>

12. `LockSupport.park()` vs `Object.wait()`

	<code>LockSupport.park()</code>	<code>Object.wait()</code>
Requires monitor	✗	✓ Must be in <code>synchronized</code>
Pre-permit (unpark before park)	✓ Works	✗ Signal lost
Spurious wakeup handling	Required	Required
Used by	AQS, <code>ReentrantLock</code>	Traditional monitors
Flexibility	Higher	Lower

13. `AtomicReference` vs `AtomicStampedReference`

	<code>AtomicReference</code>	<code>AtomicStampedReference</code>
ABA problem	✗ Vulnerable	✓ Solved
CAS checks	Value only	Value + stamp (version)
Overhead	Lower	Slightly higher
Use case	General CAS	Lock-free structures with ABA risk

14. Deadlock vs Livelock vs Starvation

	Deadlock	Livelock	Starvation
Progress	✗ None	✗ None (but active)	✗ For some threads

	Deadlock	Livelock	Starvation
Thread state	BLOCKED	RUNNABLE (busy retrying)	RUNNABLE but never scheduled
CPU usage	Low	High	Normal
Cause	Circular lock wait	Threads reacting to each other	Unfair scheduling / priority
Fix	Lock ordering, tryLock	Random backoff	Fair locks, priority tuning

15. Race Condition vs Data Race

	Race Condition	Data Race
Definition	Bug from timing dependency	Unsynchronized concurrent memory access
Requires unsync'd access	✗ Not necessarily	✓ By definition
JMM violation	✗	✓
Detectable by tools	Hard	Yes (thread sanitizers)
Can exist without the other	✓	✓

3/8/2026, 1:28:32 PM

3/14/2026, 7:00:37 AM

Here are **JPMC-style Java Concurrency Interview Questions & Answers** for 8 years of experience. JPMC focuses heavily on real-world scenarios, low-latency systems, and production-grade design.

JPMC Core Concurrency Questions

1. In a trading system, you have a shared order book. Multiple threads read and write to it. How would you design thread-safe access?

This is a classic JPMC scenario. The approach depends on read/write ratio:

Option 1 — `StampedLock` (best for read-heavy order book):

```
StampedLock lock = new StampedLock();

public Order getBestBid() {
    long stamp = lock.tryOptimisticRead();
    Order bid = bestBid; // read without lock
    if (!lock.validate(stamp)) { // someone wrote during read
        stamp = lock.readLock();
        try { bid = bestBid; }
        finally { lock.unlockRead(stamp); }
    }
    return bid;
}

public void addOrder(Order order) {
    long stamp = lock.writeLock();
    try { orderBook.add(order); }
    finally { lock.unlockWrite(stamp); }
}
```

Option 2 — For ultra-low latency: Use a **single-writer principle** — only one dedicated thread writes to the order book (LMAX Disruptor pattern), all others just read. Eliminates all locking.

2. How does the LMAX Disruptor work and why is it faster than `BlockingQueue`?

The **Disruptor** is a lock-free, inter-thread messaging framework used heavily in financial systems (including JPMC).

Key design:

- Uses a **ring buffer** (pre-allocated, fixed-size array) — no GC pressure, no object allocation
- **Single writer principle** — one producer or multiple producers with CAS
- Consumers track their own sequence number — no queue pop needed
- Uses **memory barriers** (`volatile` reads/writes) instead of locks
- Eliminates **false sharing** via cache line padding on sequence counters

Why faster than `BlockingQueue` :

- No lock/unlock overhead
- No node object allocation per message (unlike `LinkedBlockingQueue`)
- Cache-friendly sequential memory access
- Multiple consumers can read the same event without coordination (fan-out for free)

3. You have 1000 concurrent requests hitting a downstream service limited to 50 concurrent calls. How do you implement this?

```
public class DownstreamGateway {
    private final Semaphore semaphore = new Semaphore(50, true);
    private final ExecutorService executor =
        Executors.newFixedThreadPool(50);

    public CompletableFuture<Response> call(Request req) {
        return CompletableFuture.supplyAsync(() -> {
            try {
                semaphore.acquire();
                try {
                    return downstreamService.call(req);
                } finally {
                    semaphore.release();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                throw new RuntimeException(e);
            }
        }, executor);
    }
}
```

```
}  
}
```

For production at JPMC scale, also add:

- **Timeout:** `semaphore.tryAcquire(500, MILLISECONDS)` — fail fast rather than queue forever
- **Circuit breaker** (Resilience4j) to stop calling when downstream is degraded
- **Bulkhead pattern** to isolate thread pools per downstream service

4. What happens before guarantee when using `volatile` in a financial context?

In a trading system:

```
volatile boolean marketOpen = false;  
Map<String, Price> prices = new HashMap<>(); // NOT thread-safe  
  
// Thread 1 (market data thread)  
prices.put("AAPL", new Price(150.0));  
marketOpen = true; // volatile write  
  
// Thread 2 (trading thread)  
if (marketOpen) { // volatile read  
    Price p = prices.get("AAPL"); // SAFE - JMM guarantees visibility  
}
```

The `volatile` write to `marketOpen` happens-before the `volatile` read. All writes made before the volatile write (including `prices.put`) are visible to Thread 2 after it reads `marketOpen = true`.

This is a **safe publication** pattern used in financial systems to publish a snapshot of data atomically without locks.

5. A thread pool has 10 threads. A task submits a subtask to the same pool and calls `get()`. What happens?

Thread pool deadlock / pool exhaustion:

```
ExecutorService pool = Executors.newFixedThreadPool(10);  
  
// Task submitted to pool  
pool.submit(() -> {
```

```
Future<String> subtask = pool.submit(() -> "result"); // subtask queued
return subtask.get(); // DEADLOCK if all 10 threads are doing this
});
```

If all 10 threads are waiting on subtasks that are stuck in the queue, none can execute — **deadlock by starvation**.

JPMC-style fix options:

1. Use a **separate thread pool** for subtasks
2. Use `ForkJoinPool` which supports **work stealing** and **managed blocking** (`ManagedBlocker`)
3. Use `CompletableFuture` with `thenCompose` to avoid blocking entirely
4. Design tasks to be non-blocking — never call `get()` inside a pool thread

6. How would you implement a thread-safe cache with expiry in Java?

```
public class ExpiringCache<K, V> {
    private final ConcurrentHashMap<K, CacheEntry<V>> map =
        new ConcurrentHashMap<>();
    private final ScheduledExecutorService cleaner =
        Executors.newSingleThreadScheduledExecutor();
    private final long ttlMillis;

    public ExpiringCache(long ttlMillis) {
        this.ttlMillis = ttlMillis;
        cleaner.scheduleAtFixedRate(this::evictExpired,
            ttlMillis, ttlMillis, MILLISECONDS);
    }

    public V get(K key) {
        CacheEntry<V> entry = map.get(key);
        if (entry == null || entry.isExpired()) {
            map.remove(key);
            return null;
        }
        return entry.value;
    }

    public void put(K key, V value) {
        map.put(key, new CacheEntry<>(value,
            System.currentTimeMillis() + ttlMillis));
    }
}
```

```

private void evictExpired() {
    map.entrySet().removeIf(e -> e.getValue().isExpired());
}

static class CacheEntry<V> {
    final V value;
    final long expiryTime;
    CacheEntry(V value, long expiryTime) {
        this.value = value;
        this.expiryTime = expiryTime;
    }
    boolean isExpired() {
        return System.currentTimeMillis() > expiryTime;
    }
}
}

```

Production note: JPMC would likely use **Caffeine** cache which handles expiry, eviction, and concurrency far more efficiently using a window-TinyLFU algorithm.

7. What is `ForkJoinPool` and how does work stealing work?

`ForkJoinPool` is designed for **divide-and-conquer parallelism**:

```

ForkJoinPool pool = new ForkJoinPool(4);
pool.invoke(new RecursiveTask<Long>() {
    protected Long compute() {
        if (taskSmallEnough()) return computeDirectly();
        RecursiveTask<Long> left = new SubTask(leftHalf);
        left.fork(); // submit to pool asynchronously
        Long rightResult = new SubTask(rightHalf).compute();
        return left.join() + rightResult;
    }
});

```

Work stealing:

- Each thread has its own **deque** of tasks
- Thread pushes/pops from its own deque head (**LIFO** — cache friendly)
- Idle threads steal from the **tail** of busy threads' deques (**FIFO** — takes oldest tasks)

- This minimizes contention — owner and stealer operate at opposite ends

Used in **parallel streams**, `CompletableFuture.supplyAsync()`, and recursive financial computations (e.g., risk calculations).

8. How do you prevent a memory leak with `ThreadLocal` in a thread pool?

`ThreadLocal` stores a value per thread. In a thread pool, threads are **reused** — so `ThreadLocal` values persist across tasks:

```
// LEAK — ThreadLocal never cleaned up
private static ThreadLocal<HeavyObject> local =
    new ThreadLocal<>();

pool.submit(() -> {
    local.set(new HeavyObject()); // set for this task
    doWork();
    // thread returns to pool — HeavyObject stays in memory!
});
```

Fix — always clean up in a finally block:

```
pool.submit(() -> {
    try {
        local.set(new HeavyObject());
        doWork();
    } finally {
        local.remove(); // CRITICAL — prevents leak
    }
});
```

At JPMC, this is a common cause of **memory leaks in app servers** (Tomcat, JBoss) where thread pools reuse threads across requests. Also dangerous with `InheritableThreadLocal` across parent/child thread contexts.

9. Explain `ConcurrentHashMap.compute()` vs `putIfAbsent()` vs `merge()`.

All three are atomic operations — but serve different purposes:

`putIfAbsent(key, value)` — insert only if key doesn't exist. Value computed **before** the call (wasteful if expensive):

```
map.putIfAbsent("AAPL", new ArrayList<>()); // list created even if key exists
```

`computeIfAbsent(key, fn)` — insert only if absent, value computed **lazily**:

```
map.computeIfAbsent("AAPL", k -> new ArrayList<>()).add(order);
```

`compute(key, fn)` — atomically update for existing or new key:

```
map.compute("AAPL", (k, v) -> v == null ? 1 : v + 1); // atomic counter
```

`merge(key, value, fn)` — if absent, puts value; if present, applies merge function:

```
map.merge("AAPL", 1, Integer::sum); // cleanest atomic counter
```

In JPMC context, `computeIfAbsent` is used heavily for grouping orders by symbol, `merge` for aggregating positions.

10. How would you implement a producer-consumer for real-time market data with minimal latency?

For JPMC-level low latency:

```
// Use ArrayBlockingQueue with bounded capacity for back-pressure
BlockingQueue<MarketData> queue = new ArrayBlockingQueue<>(1000);

// Producer – market data feed handler
Thread producer = new Thread(() -> {
    while (running) {
        MarketData tick = feed.nextTick();
        if (!queue.offer(tick, 1, MILLISECONDS)) {
            droppedTicksMeter.mark(); // monitor drops
        }
    }
});

// Consumer – strategy engine
Thread consumer = new Thread(() -> {
    while (running) {
```

```

    MarketData tick = queue.poll(100, MILLISECONDS);
    if (tick != null) strategy.onTick(tick);
}
});

```

For **ultra-low latency** (sub-microsecond):

- Replace `BlockingQueue` with **LMAX Disruptor** ring buffer
- Use **busy-spin wait strategy** instead of blocking
- Pin threads to CPU cores (via JNA/JNI)
- Use **off-heap memory** to avoid GC pauses

11. What is `volatile` not sufficient for? Give a JPMC-relevant example.

`volatile` guarantees visibility but **not atomicity for compound operations**.

```

volatile long position = 0;

// Thread 1
position += 100; // READ then WRITE – not atomic!

// Thread 2
position -= 50; // race condition – one update may be lost

```

`position += 100` compiles to: read position → add 100 → write position. Two threads can read the same value simultaneously, both add their delta, and one update is lost — a **lost update** bug. In a trading system, this means incorrect position tracking.

Fix: Use `AtomicLong`:

```

AtomicLong position = new AtomicLong(0);
position.addAndGet(100); // atomic CAS loop internally

```

Or `LongAdder` for high-contention counters (e.g., trade count metrics) — even faster under contention by distributing across cells.

12. How do you handle `InterruptedException` correctly?

This is a very common JPMC interview question. Two correct approaches:

Option 1 — Propagate (preferred in library code):

```
public void doWork() throws InterruptedException {
    queue.take(); // just let InterruptedException propagate
}
```

Option 2 — Restore interrupt flag (when you can't propagate):

```
public void doWork() {
    try {
        queue.take();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // CRITICAL – restore the flag
        // handle graceful shutdown
    }
}
```

What NOT to do:

```
catch (InterruptedException e) {
    // swallowing it – WRONG
}
```

Swallowing `InterruptedException` breaks thread pool shutdown, application stop sequences, and any cooperative cancellation mechanism. In a trading system, this can prevent graceful market-close shutdown.

13. Explain `AbstractQueuedSynchronizer (AQS)` — the backbone of Java locks.

AQS is the foundation of `ReentrantLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, and more.

Core idea:

- Maintains an **int state** (e.g., 0 = unlocked, 1 = locked, N = semaphore permits)
- Maintains a **CLH queue** (virtual doubly-linked list) of waiting threads
- Subclasses implement `tryAcquire(int)` and `tryRelease(int)` with CAS on state

```
Thread tries acquire → CAS on state
├─ SUCCESS → proceeds
```

```
└─ FAIL → enqueued as Node in CLH queue
      parked via LockSupport.park()
      unparked when predecessor releases
```

Exclusive mode (ReentrantLock) vs Shared mode (Semaphore/ReadWriteLock) both implemented via AQS.

Why it matters at JPMC: understanding AQS means you understand **why** `ReentrantLock` is fair/unfair, what the overhead actually is, and when to bypass it entirely with CAS-based structures.

14. How do `parallel streams` relate to concurrency? What are the risks?

Parallel streams use `ForkJoinPool.commonPool()` internally:

```
orders.parallelStream()
    .filter(o -> o.getValue() > 1_000_000)
    .map(this::enrichOrder)
    .collect(toList());
```

Risks at JPMC:

1. **Shared common pool:** Parallel streams, `CompletableFuture.supplyAsync()`, and any other `ForkJoinPool` usage share the same pool. One slow operation starves all others.
2. **Blocking operations:** If `enrichOrder` does a DB call, the common pool threads block — dangerous as pool is sized to CPU cores.
3. **Order non-determinism:** Results are not ordered unless you use `forEachOrdered` or `collect`.
4. **Thread-safety of operations:** The lambda must be stateless and non-interfering.

Fix for blocking: Use a custom pool:

```
ForkJoinPool customPool = new ForkJoinPool(20);
customPool.submit(() ->
    orders.parallelStream().map(this::enrichOrder).collect(toList())
).get();
```

15. Design a thread-safe counter for high-frequency trade tracking. Compare options.

Option	Throughput	Use case
<code>synchronized int</code>	Low	Single-threaded-like
<code>AtomicLong</code>	Medium	Moderate contention
<code>LongAdder</code>	Highest	High contention counters
<code>LongAccumulator</code>	Highest	Custom aggregation

```
// LongAdder – best for high-frequency incrementing
LongAdder tradeCount = new LongAdder();

// Producer threads
tradeCount.increment(); // each thread updates its own cell

// Reporter thread (less frequent)
long total = tradeCount.sum(); // sums all cells
```

`LongAdder` internally maintains a `Cell[]` array — each thread updates its own cell under contention, eliminating CAS retries. `sum()` adds all cells. The trade-off: `sum()` is not a precise snapshot — fine for metrics, not for exact accounting.

For **exact atomic accounting** (e.g., P&L), use `AtomicLong` or `synchronized`.

JPMC Scenario-Based Questions

16. A critical batch job runs nightly. Midway, it hangs. How do you investigate?

1. **Take thread dump** — `jstack <pid>` or `kill -3 <pid>`
2. Look for threads in **BLOCKED** state — find who holds the lock they're waiting for
3. Look for **WAITING** threads — likely `Object.wait()` or `LockSupport.park()` — check the stack trace for what condition they're waiting on
4. Look for **deadlock section** — JVM prints it automatically
5. Check **I/O threads** — DB connection pool exhausted? Network call hung?
6. Use **JFR** if enabled — look at lock contention events leading up to the hang
7. Check thread pool queue size via JMX — is the queue full? Are tasks being rejected?
8. Check GC logs — a full GC pause can look like a hang

17. How would you implement a distributed lock in a microservices environment at JPMC?

Pure Java concurrency isn't enough across JVMs. Options:

Redis-based (Redisson):

```
RLock lock = redissonClient.getLock("ORDER_LOCK_" + orderId);
try {
    if (lock.tryLock(5, 30, TimeUnit.SECONDS)) {
        processOrder(orderId);
    }
} finally {
    lock.unlock();
}
```

Database-based: Use `SELECT FOR UPDATE` or a lock table with heartbeat.

ZooKeeper-based: Strong consistency guarantees, but higher latency.

Key concerns for JPMC:

- **Lock expiry:** Always set TTL — process crash must auto-release the lock
- **Fencing token:** Prevent stale lock holders from making writes after lock expiry
- **Idempotency:** Even with distributed locks, design operations to be idempotent as a safety net

18. How do you make a `HashMap` thread-safe in different ways and what are the trade-offs?

Approach	Thread Safety	Performance	Notes
<code>Hashtable</code>	✓ Full sync	Very poor	Legacy, avoid
<code>Collections.synchronizedMap()</code>	✓ Full sync	Poor	Every op locks whole map
<code>ConcurrentHashMap</code>	✓ Partial lock	Excellent	Lock per bucket, lock-free reads
<code>CopyOnWriteMap</code> (no built-in)	✓	Poor for writes	Read-heavy only
Immutable <code>Map.of()</code>	✓ Read-only	Excellent	Only if no writes needed

JPMC answer: Always use `ConcurrentHashMap`. Use `Map.of()` for configuration maps loaded once at startup. Never use `Hashtable` in new code.

Additional: for read-heavy reference data (e.g., instrument master), load once into an immutable `Map` and republish atomically via `AtomicReference`:

```
AtomicReference<Map<String, Instrument>> ref = new AtomicReference<>(load());  
// Refresh: ref.set(Collections.unmodifiableMap(newData));  
// Read: ref.get().get(symbol); // always lock-free
```

3/14/2026, 7:02:12 AM

What is a Bucket in HashMap?

Simple Definition

A **bucket** is just an **index slot in an internal array** inside a `HashMap`. When you put a key-value pair, HashMap decides **which slot (bucket) to store it in** using the key's hash code.

Internal Structure

```
HashMap internal array (table[]):
```

```
Index 0 → [ ]
Index 1 → [ "Alice"=25 ] → [ "Bob"=30 ] ← bucket 1 has 2 entries (collision)
Index 2 → [ ]
Index 3 → [ "Charlie"=35 ]
Index 4 → [ ]
...
Index 15 → [ "Dave"=40 ]
```

Each index in the array = **one bucket**.

How a Key Lands in a Bucket

```
map.put("Alice", 25);
```

Internally HashMap does:

```
1. hash = "Alice".hashCode()           // e.g., 63476538
2. index = hash & (array.length - 1)    // e.g., 63476538 & 15 = 10
3. Store at table[10]                  // bucket 10
```

So the bucket index is determined by **hash % array size** (using bitwise AND for performance).

What Happens Inside a Bucket

Each bucket can hold **multiple entries** when two keys hash to the same index — this is called a **collision**.

Java 7 and before — bucket stores a **linked list**:

```
bucket[1] → Node("Alice", 25) → Node("Bob", 30) → null
```

Java 8 onwards — bucket starts as a **linked list**, but converts to a **red-black tree** when entries exceed **8 nodes** (treeification threshold):

```
bucket[1] (linked list, ≤ 8 entries):  
→ Node("Alice") → Node("Bob") → Node("Carol") ...
```

```
bucket[1] (tree, > 8 entries):  
      Node("Dave")  
     /      \  
Node("Bob") Node("Frank")  
  /  
Node("Alice")
```

This changes worst-case lookup from **O(n)** to **O(log n)** for heavily collided buckets.

Bucket Lifecycle — Step by Step

```
HashMap<String, Integer> map = new HashMap<>(); // 16 buckets created  
  
map.put("Alice", 25); // hash("Alice") → bucket 3  
map.put("Bob", 30); // hash("Bob") → bucket 7  
map.put("Carol", 35); // hash("Carol") → bucket 3 ← collision with Alice!
```

At bucket 3:

```
bucket[3] → ["Alice"]=25 → ["Carol"]=35 // linked list
```

When you do `map.get("Carol")`:

1. `hash("Carol")` → bucket 3
2. Walk linked list at bucket 3
3. Compare each key using `.equals()`
4. Found "Carol" → return 35

Rehashing and Bucket Count

- Default initial capacity = **16 buckets**
- Default load factor = **0.75**
- When entries exceed `16 × 0.75 = 12`, the array **doubles to 32 buckets**
- All existing entries are **rehashed** into new bucket positions
- This is called **rehashing** — expensive $O(n)$ operation

Before rehash (16 buckets):

`bucket[1]` → Alice → Bob → Carol (3 entries, overloaded)

After rehash (32 buckets):

`bucket[1]` → Alice

`bucket[17]` → Bob

`bucket[1]` → Carol (better distributed)

Why Buckets Matter for Performance

Scenario	Bucket behavior	Performance
Good hash, even spread	1 entry per bucket	$O(1)$ get/put
Many collisions (bad hash)	Long linked list	$O(n)$ get/put
Many collisions (Java 8+)	Tree after 8 entries	$O(\log n)$ get/put
Too many entries (load factor exceeded)	Rehash triggered	$O(n)$ one-time cost

Real Problem — All Keys in One Bucket

If `hashCode()` always returns the same value:

```
class BadKey {  
    public int hashCode() { return 1; } // all keys → bucket 1  
}
```

Every entry goes to **bucket 1** — HashMap degrades to a linked list, $O(n)$ for everything. This is why a **good** `hashCode()` **implementation matters** — it should distribute keys evenly across buckets.

Quick Summary

- **Bucket** = one slot in HashMap's internal array
- Bucket index is derived from `hashCode()`
- One bucket can hold multiple entries (collision → linked list → tree in Java 8)
- More buckets = less collision = better performance
- HashMap grows (rehashes) when too full to keep buckets sparse

3/14/2026, 7:05:20 AM

 What is this things

ReentrantLock, Semaphore, CountDownLatch, CyclicBarrier

3/14/2026, 7:15:12 AM

Great question! Let me explain each one simply with real-world analogies and code.

1. **ReentrantLock**

Simple Definition

A **manual lock** that only **one thread can hold at a time**. Like a **toilet key** — only one person holds it, others wait outside.

"Reentrant" means

The **same thread** can lock it **multiple times** without deadlocking itself — it just needs to unlock the same number of times.

```
ReentrantLock lock = new ReentrantLock();

lock.lock();    // Thread 1 acquires lock
lock.lock();    // Same thread acquires again – allowed! (count = 2)
lock.unlock();  // count = 1
lock.unlock();  // count = 0 – fully released
```

Real Example

```
ReentrantLock lock = new ReentrantLock();

public void transferMoney(Account from, Account to, int amount) {
    lock.lock();           // only one thread can transfer at a time
    try {
        from.debit(amount);
        to.credit(amount);
    } finally {
        lock.unlock();     // ALWAYS unlock in finally
    }
}
```

Key Features

```
lock.tryLock()           // try to acquire, don't wait – returns true/false
lock.tryLock(5, TimeUnit.SECONDS) // wait max 5 seconds
lock.lockInterruptibly() // can be interrupted while waiting
new ReentrantLock(true)  // fair lock – threads served in order
```

vs `synchronized`

	<code>synchronized</code>	<code>ReentrantLock</code>
Auto unlock	✓	✗ Manual
Try without blocking	✗	✓ <code>tryLock()</code>
Fairness	✗	✓
Timeout	✗	✓

2. `Semaphore`

Simple Definition

A **counter of permits**. Controls how many threads can do something **simultaneously**. Like a **parking lot with N spaces** — only N cars allowed in at once, others wait outside.

Real-World Analogy

A restaurant has only **5 tables**. When all 5 are occupied, new customers wait. When one table frees up, the next customer gets in.

```
Semaphore semaphore = new Semaphore(5); // 5 permits

// Customer arrives
semaphore.acquire(); // takes 1 permit (if 0 permits → waits)
try {
    eatFood(); // uses the table
} finally {
    semaphore.release(); // returns 1 permit
}
```

Real Example — Connection Pool

```
Semaphore semaphore = new Semaphore(10); // max 10 DB connections

public void queryDatabase() throws InterruptedException {
    semaphore.acquire();           // wait for a free connection
    try {
        db.executeQuery();
    } finally {
        semaphore.release();      // free the connection
    }
}
```

Key Methods

```
semaphore.acquire()           // take 1 permit, block if none
semaphore.acquire(3)          // take 3 permits at once
semaphore.tryAcquire()        // try, don't block
semaphore.tryAcquire(2, SECONDS) // try for 2 seconds
semaphore.release()           // return 1 permit
semaphore.availablePermits()  // how many permits free right now
```

Special Case — Semaphore(1)

A Semaphore with **1 permit** acts like a **mutex (mutual exclusion lock)** — similar to `synchronized`.

3. `CountDownLatch`

Simple Definition

A **one-time gate**. One or more threads **wait** until a count reaches **zero**. Like a **rocket launch** — wait for all systems (engines, fuel, navigation) to be ready, then launch.

Real-World Analogy

A **race** doesn't start until all **5 runners** are at the starting line. The starter waits for count = 0.

```
CountDownLatch latch = new CountDownLatch(5); // wait for 5 events

// 5 worker threads
for (int i = 0; i < 5; i++) {
    new Thread(() -> {
```

```

        doWork();
        latch.countDown(); // signal "I'm done" — count goes 5→4→3→2→1→0
    }).start();
}

latch.await(); // MAIN THREAD WAITS HERE until count = 0
System.out.println("All workers done! Proceed.");

```

Real Example — App Startup

```

CountDownLatch startupLatch = new CountDownLatch(3); // 3 services to start

new Thread(() -> { startDatabase(); startupLatch.countDown(); }).start();
new Thread(() -> { startCache(); startupLatch.countDown(); }).start();
new Thread(() -> { startMessaging(); startupLatch.countDown(); }).start();

startupLatch.await(); // wait for all 3
System.out.println("All services up! App is ready.");

```

Key Points

- Count can only go **DOWN**, never reset
- `countDown()` never blocks — just decrements
- `await()` blocks until count = 0
- One-time use** — cannot be reused after reaching 0

```

latch.countDown()    // decrement count by 1
latch.await()        // block until count = 0
latch.await(5, SECONDS) // block max 5 seconds
latch.getCount()     // current count

```

4. `CyclicBarrier` 🚧

Simple Definition

All threads **wait at a barrier** until **everyone arrives**, then they **all proceed together**. Like a **group of friends** waiting for each other before entering a movie — nobody goes in until the whole group is there.

Real-World Analogy

A **relay race** — the next leg doesn't start until all runners finish the current leg. Then everyone starts the next leg **together**.

```
CyclicBarrier barrier = new CyclicBarrier(3); // wait for 3 threads

Runnable task = () -> {
    doPhase1Work();
    barrier.await(); // wait for all 3 threads to finish phase 1
    // ALL 3 threads proceed together from here
    doPhase2Work();
    barrier.await(); // wait again for phase 2
    doPhase3Work();
};

new Thread(task).start();
new Thread(task).start();
new Thread(task).start();
```

Real Example — Parallel Data Processing

```
CyclicBarrier barrier = new CyclicBarrier(4, () -> {
    System.out.println("All chunks processed! Merging results..."); // runs when all
    arrive
});

// 4 threads each process a chunk of data
for (int i = 0; i < 4; i++) {
    final int chunkId = i;
    new Thread(() -> {
        processChunk(chunkId); // each thread processes its chunk
        barrier.await();       // wait for all 4 to finish
        mergeResults();         // all proceed together
    }).start();
}
```

Key Difference from **CountDownLatch**

	CountDownLatch	CyclicBarrier
Reusable	❌ One-time only	✅ Resets automatically
Who waits	Main thread waits for workers	All threads wait for each other
Count direction	Counts DOWN	Counts UP to barrier

CountDownLatch

CyclicBarrier

Barrier action

✗ None

✓ Optional runnable on trip

🎯 All 4 Together — Simple Analogy

Class	Real-World Analogy	Purpose
ReentrantLock	Toilet key 🚽 — one person at a time	Mutual exclusion
Semaphore	Parking lot 🅑 — N cars allowed	Limit concurrent access
CountDownLatch	Rocket launch 🚀 — wait for all checks	Wait for N events once
CyclicBarrier	Group photo 📸 — wait for everyone	All threads sync at a point, repeat

Quick Code Comparison

```
// ReentrantLock — one thread at a time
ReentrantLock lock = new ReentrantLock();
lock.lock(); doWork(); lock.unlock();

// Semaphore — N threads at a time
Semaphore sem = new Semaphore(3);
sem.acquire(); doWork(); sem.release();

// CountDownLatch — wait for N things to finish
CountDownLatch latch = new CountDownLatch(3);
latch.countDown(); // called 3 times by workers
latch.await();     // main thread waits

// CyclicBarrier — all threads meet, then continue together
CyclicBarrier barrier = new CyclicBarrier(3);
barrier.await(); // each of 3 threads calls this — all proceed when all arrive
```

3/14/2026, 7:15:49 AM

4/23/2026, 8:31:13 AM

Here are **MBB Labs (McKinsey, Bain, BCG style tech arm) Concurrency Scenario-Based Questions & Answers** — these focus on **real-world problem solving, system design thinking, and deep technical reasoning**.

SCENARIO 1: Bank Transaction System

Scenario: Two users simultaneously transfer money from the same account. How do you prevent incorrect balance?

Problem:

```
// UNSAFE
public void transfer(Account from, Account to, int amount) {
    if (from.getBalance() >= amount) { // Thread 1 checks: 1000 >= 500 ✓
        // Thread 2 checks: 1000 >= 700 ✓
        from.debit(amount); // Both threads debit – balance goes
        // negative!
        to.credit(amount);
    }
}
```

Fix 1 — ReentrantLock with ordered locking (prevents deadlock):

```
public void transfer(Account from, Account to, int amount) {
    // Always lock lower ID first – prevents deadlock
    Account first = from.getId() < to.getId() ? from : to;
    Account second = from.getId() < to.getId() ? to : from;

    synchronized (first) {
        synchronized (second) {
            if (from.getBalance() >= amount) {
                from.debit(amount);
                to.credit(amount);
            }
        }
    }
}
```

```
}  
}
```

Fix 2 — Database-level with optimistic locking:

```
@Version  
private int version; // JPA optimistic lock  
  
// If two threads update same row, one gets OptimisticLockException  
// Retry logic handles it cleanly
```

Key insight: Always acquire locks in a **consistent global order** to prevent deadlock. Never lock Account A then B in one thread and B then A in another.

SCENARIO 2: Flight Booking System

Scenario: 500 users simultaneously try to book the last seat on a flight. How do you ensure only one succeeds?

Problem — classic race condition:

```
// UNSAFE — both threads see availableSeats = 1  
public boolean bookSeat(int flightId) {  
    int seats = getAvailableSeats(flightId); // Thread 1: seats = 1  
                                           // Thread 2: seats = 1  
  
    if (seats > 0) {  
        decrementSeats(flightId); // Both decrement → seats = -1!  
        return true;  
    }  
    return false;  
}
```

Fix 1 — Atomic CAS approach:

```
ConcurrentHashMap<Integer, AtomicInteger> seatMap = new ConcurrentHashMap<>();  
  
public boolean bookSeat(int flightId) {  
    AtomicInteger seats = seatMap.get(flightId);  
    while (true) {
```

```

    int current = seats.get();
    if (current <= 0) return false; // no seats
    if (seats.compareAndSet(current, current - 1)) {
        return true; // CAS succeeded – booked!
    }
    // CAS failed – another thread booked, retry
}
}

```

Fix 2 — Database pessimistic lock:

```

SELECT * FROM flights WHERE id = ? FOR UPDATE; -- locks the row
UPDATE flights SET seats = seats - 1 WHERE id = ? AND seats > 0;

```

Fix 3 — Semaphore per flight:

```

ConcurrentHashMap<Integer, Semaphore> flightSemaphores = new ConcurrentHashMap<>();

public boolean bookSeat(int flightId, int totalSeats) {
    Semaphore semaphore = flightSemaphores.computeIfAbsent(
        flightId, k -> new Semaphore(totalSeats)
    );
    return semaphore.tryAcquire(); // atomic – only N threads succeed
}

```

SCENARIO 3: Real-Time Dashboard

Scenario: A dashboard shows live stock prices updated by one thread and read by thousands of UI threads. How do you design this?

Wrong approach:

```

// Synchronized – causes massive read contention
private synchronized Price getPrice(String symbol) {
    return prices.get(symbol);
}

```

Correct approach — lock-free reads with volatile publication:

```

public class PriceFeed {
    // AtomicReference for safe publication of entire snapshot
    private final AtomicReference<Map<String, Price>> snapshot =
        new AtomicReference<>(Collections.emptyMap());

    // Called by ONE writer thread
    public void updatePrice(String symbol, Price price) {
        snapshot.updateAndGet(current -> {
            Map<String, Price> updated = new HashMap<>(current);
            updated.put(symbol, price);
            return Collections.unmodifiableMap(updated);
        });
    }

    // Called by THOUSANDS of reader threads – ZERO locking
    public Price getPrice(String symbol) {
        return snapshot.get().get(symbol); // lock-free read
    }
}

```

Why this works:

- **AtomicReference** guarantees the snapshot is **atomically published**
- Readers always see a **complete, consistent snapshot** — never a partial update
- No reader ever blocks another reader
- Writer creates a new map each time — readers referencing old map still safe (immutable)

SCENARIO 4: Rate Limiter for API Gateway

Scenario: Your API gateway allows max 100 requests per second per user. Design a thread-safe rate limiter.

Token Bucket implementation:

```

public class RateLimiter {
    private final int maxTokens;
    private final long refillIntervalNanos;
    private AtomicLong availableTokens;
    private volatile long lastRefillTime;
}

```

```

public RateLimiter(int requestsPerSecond) {
    this.maxTokens = requestsPerSecond;
    this.availableTokens = new AtomicLong(requestsPerSecond);
    this.lastRefillTime = System.nanoTime();
    this.refillIntervalNanos = 1_000_000_000L / requestsPerSecond;
}

public boolean tryAcquire() {
    refill();
    while (true) {
        long tokens = availableTokens.get();
        if (tokens <= 0) return false;           // rate limit exceeded
        if (availableTokens.compareAndSet(tokens, tokens - 1)) {
            return true;                         // token consumed
        }
        // CAS failed – retry
    }
}

private void refill() {
    long now = System.nanoTime();
    long elapsed = now - lastRefillTime;
    long tokensToAdd = elapsed / refillIntervalNanos;
    if (tokensToAdd > 0) {
        availableTokens.updateAndGet(
            current -> Math.min(maxTokens, current + tokensToAdd)
        );
        lastRefillTime = now;
    }
}

// Per-user rate limiter
ConcurrentHashMap<String, RateLimiter> userLimiters = new ConcurrentHashMap<>();

public boolean isAllowed(String userId) {
    RateLimiter limiter = userLimiters.computeIfAbsent(
        userId, k -> new RateLimiter(100)
    );
    return limiter.tryAcquire();
}

```

SCENARIO 5: Parallel Report Generation

Scenario: A report needs data from 5 different services (DB, cache, REST APIs). Each takes 2 seconds. How do you fetch all in parallel and merge?

Wrong — sequential (10 seconds total):

```
Data db      = fetchFromDB();      // 2 sec
Data cache   = fetchFromCache();   // 2 sec
Data api1    = fetchFromAPI1();    // 2 sec
Data api2    = fetchFromAPI2();    // 2 sec
Data api3    = fetchFromAPI3();    // 2 sec
// Total: 10 seconds
```

Correct — parallel with `CompletableFuture` (2 seconds total):

```
ExecutorService executor = Executors.newFixedThreadPool(5);

CompletableFuture<Data> dbFuture      = CompletableFuture.supplyAsync(() ->
    fetchFromDB(),      executor);
CompletableFuture<Data> cacheFuture = CompletableFuture.supplyAsync(() ->
    fetchFromCache(),   executor);
CompletableFuture<Data> api1Future  = CompletableFuture.supplyAsync(() ->
    fetchFromAPI1(),    executor);
CompletableFuture<Data> api2Future  = CompletableFuture.supplyAsync(() ->
    fetchFromAPI2(),    executor);
CompletableFuture<Data> api3Future  = CompletableFuture.supplyAsync(() ->
    fetchFromAPI3(),    executor);

// Wait for all 5 to complete
CompletableFuture.allOf(dbFuture, cacheFuture, api1Future, api2Future, api3Future)
    .thenApply(v -> mergeReport(
        dbFuture.join(),
        cacheFuture.join(),
        api1Future.join(),
        api2Future.join(),
        api3Future.join()
    ))
    .exceptionally(ex -> {
        log.error("Report generation failed", ex);
        return fallbackReport();
    });
```

```
});  
// Total: ~2 seconds (longest single call)
```

SCENARIO 6: Cache Stampede Problem

Scenario: A cached value expires. 1000 threads simultaneously find cache empty and all hit the DB at once. How do you fix this?

This is called **cache stampede** or **thundering herd**.

Fix 1 — Double-checked locking with ConcurrentHashMap:

```
private final ConcurrentHashMap<String, String> cache = new ConcurrentHashMap<>();  
private final ConcurrentHashMap<String, Object> locks = new ConcurrentHashMap<>();  
  
public String getValue(String key) {  
    String value = cache.get(key);  
    if (value != null) return value;    // fast path – no lock  
  
    // Only one thread per key loads from DB  
    Object keyLock = locks.computeIfAbsent(key, k -> new Object());  
    synchronized (keyLock) {  
        value = cache.get(key);    // check again inside lock  
        if (value == null) {  
            value = loadFromDB(key);    // only ONE thread hits DB  
            cache.put(key, value);  
        }  
    }  
    locks.remove(key);  
    return value;  
}
```

Fix 2 — Probabilistic early expiration (advanced): Refresh the cache **before** it expires based on probability, so stampede never happens.

Fix 3 — Semaphore per key:

```
ConcurrentHashMap<String, Semaphore> keyLocks = new ConcurrentHashMap<>();
```

```

public String getValue(String key) {
    String cached = cache.get(key);
    if (cached != null) return cached;

    Semaphore sem = keyLocks.computeIfAbsent(key, k -> new Semaphore(1));
    sem.acquire();           // only 1 thread loads, others wait
    try {
        String val = cache.get(key);
        if (val == null) {
            val = loadFromDB(key);
            cache.put(key, val);
        }
        return val;
    } finally {
        sem.release();
        keyLocks.remove(key);
    }
}

```

SCENARIO 7: Order Processing Pipeline

Scenario: Orders come in, must go through: **Validate** → **Enrich** → **Risk Check** → **Execute**. Each stage is slow. Design a concurrent pipeline.

```

public class OrderPipeline {

    // Each stage has its own bounded queue – provides back-pressure
    BlockingQueue<Order> validateQueue = new ArrayBlockingQueue<>(100);
    BlockingQueue<Order> enrichQueue  = new ArrayBlockingQueue<>(100);
    BlockingQueue<Order> riskQueue     = new ArrayBlockingQueue<>(100);
    BlockingQueue<Order> executeQueue  = new ArrayBlockingQueue<>(100);

    ExecutorService pool = Executors.newFixedThreadPool(8);

    public void start() {
        // Stage 1 – Validator (2 threads)
        for (int i = 0; i < 2; i++) {
            pool.submit(() -> {
                while (true) {

```

```

        Order order = validateQueue.take();
        if (validator.validate(order)) {
            enrichQueue.put(order);    // pass to next stage
        }
    }
});
}

// Stage 2 – Enricher (2 threads)
for (int i = 0; i < 2; i++) {
    pool.submit(() -> {
        while (true) {
            Order order = enrichQueue.take();
            enricher.enrich(order);
            riskQueue.put(order);
        }
    });
}

// Stage 3 – Risk Check (2 threads)
for (int i = 0; i < 2; i++) {
    pool.submit(() -> {
        while (true) {
            Order order = riskQueue.take();
            if (riskEngine.check(order)) {
                executeQueue.put(order);
            }
        }
    });
}

// Stage 4 – Executor (2 threads)
for (int i = 0; i < 2; i++) {
    pool.submit(() -> {
        while (true) {
            Order order = executeQueue.take();
            executor.execute(order);
        }
    });
}

}

public void submitOrder(Order order) throws InterruptedException {
    validateQueue.put(order); // entry point
}

```

```
}  
}
```

Key design points:

- Each stage is independent — stages run truly in parallel
- Bounded queues provide **back-pressure** — if risk check is slow, enricher slows down automatically
- Easy to **scale individual stages** — just add more threads to the slow stage
- This is the **pipeline pattern** — similar to how LMAX Disruptor works

SCENARIO 8: Lazy Initialization in Multithreaded Environment

Scenario: A heavy object (DB connection factory) should be created only once, only when first needed, thread-safely.

Wrong — no synchronization:

```
private HeavyObject instance;  
public HeavyObject get() {  
    if (instance == null)  
        instance = new HeavyObject(); // multiple threads create it!  
    return instance;  
}
```

Wrong — synchronized but slow:

```
public synchronized HeavyObject get() {  
    if (instance == null)  
        instance = new HeavyObject();  
    return instance; // every read locks – massive contention  
}
```

Correct — Double Checked Locking with volatile:

```
private volatile HeavyObject instance;  
  
public HeavyObject get() {  
    if (instance == null) { // check 1 – no lock, fast path  
        synchronized (this) {  
            if (instance == null)  
                instance = new HeavyObject();  
        }  
    }  
    return instance;  
}
```

```

        synchronized (this) {
            if (instance == null) {        // check 2 – inside lock
                instance = new HeavyObject();
            }
        }
    }
    return instance;
}

```

Best — Initialization on Demand Holder (simplest & safest):

```

public class HeavyObjectHolder {
    // JVM loads inner class only when getInstance() is first called
    private static class Holder {
        static final HeavyObject INSTANCE = new HeavyObject();
    }

    public static HeavyObject getInstance() {
        return Holder.INSTANCE; // class loading is thread-safe by JVM
    }
}

```

No locks, no volatile, lazy, and thread-safe. This is the **recommended pattern**.

SCENARIO 9: Thread Pool Sizing

Scenario: How do you decide thread pool size for your application?

Formula:

CPU-bound tasks:

threads = number of CPU cores + 1
(extra 1 handles the rare page fault/pause)

I/O-bound tasks:

threads = cores × (1 + wait time / compute time)

Example:

8 cores, each task spends 90% waiting for DB (wait=9ms, compute=1ms)
threads = $8 \times (1 + 9/1) = 8 \times 10 = 80$ threads

Practical approach:

```
int cores = Runtime.getRuntime().availableProcessors();

// CPU-bound (sorting, calculations, compression)
ExecutorService cpuPool = Executors.newFixedThreadPool(cores + 1);

// I/O-bound (DB calls, REST calls, file I/O)
ExecutorService ioPool = Executors.newFixedThreadPool(cores * 10);

// NEVER mix CPU and I/O tasks in the same pool
// CPU tasks get starved by blocking I/O threads
```

Always monitor and tune:

- Queue depth growing → pool too small or tasks too slow
- CPU under-utilized but queue growing → I/O bound — increase pool size
- CPU at 100% → CPU bound — more threads won't help, optimize algorithm

SCENARIO 10: Graceful Shutdown of Multi-threaded App

Scenario: Your application receives a shutdown signal. It has running threads processing orders. How do you shut down gracefully without losing data?

```
public class OrderProcessor {
    private final ExecutorService pool = Executors.newFixedThreadPool(10);
    private volatile boolean running = true;

    // Register shutdown hook
    public void init() {
        Runtime.getRuntime().addShutdownHook(new Thread(this::shutdown));
    }

    public void submitOrder(Order order) {
        if (!running) throw new IllegalStateException("Shutting down");
        pool.submit(() -> processOrder(order));
    }
}
```

```

}

private void processOrder(Order order) {
    try {
        validate(order);
        execute(order);
        audit(order);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // restore flag
        saveForRetry(order); // don't lose the order!
    }
}

public void shutdown() {
    running = false; // stop accepting new orders

    pool.shutdown(); // no new tasks accepted

    try {
        // Wait up to 60 seconds for running orders to complete
        if (!pool.awaitTermination(60, TimeUnit.SECONDS)) {
            log.warn("Forcing shutdown – some orders may be incomplete");
            List<Runnable> pending = pool.shutdownNow();
            savePendingForRetry(pending); // persist for recovery
        } else {
            log.info("Clean shutdown – all orders processed");
        }
    } catch (InterruptedException e) {
        pool.shutdownNow();
        Thread.currentThread().interrupt();
    }
}
}

```

MBB Labs Key Themes Summary

Scenario	Core Concept Used
Bank transfer	Lock ordering, deadlock prevention
Flight booking	CAS, Semaphore, optimistic lock
Live dashboard	AtomicReference, lock-free reads

Scenario	Core Concept Used
Rate limiter	Token bucket, CAS, ConcurrentHashMap
Parallel reports	CompletableFuture, allOf
Cache stampede	Double-checked locking, per-key lock
Order pipeline	BlockingQueue, pipeline pattern, back-pressure
Lazy init	Volatile DCL, Holder pattern
Thread pool sizing	CPU vs I/O bound formula
Graceful shutdown	shutdown/awaitTermination, InterruptedException

4/23/2026, 8:32:17 AM

4/23/2026, 8:32:38 AM

Here are **all the topics missed** — covering every corner of Java Multithreading & Concurrency:

1. Thread Basics (Often Skipped)

Thread Life Cycle — All States:

NEW → RUNNABLE → BLOCKED → WAITING → TIMED_WAITING → TERMINATED

```
Thread t = new Thread(() -> {});  
// NEW          - created, not started  
t.start();  
// RUNNABLE     - running or ready to run  
// BLOCKED      - waiting for monitor lock (synchronized)  
// WAITING       - Object.wait(), Thread.join(), LockSupport.park()  
// TIMED_WAITING - Thread.sleep(ms), wait(ms), tryLock(ms)  
// TERMINATED   - run() completed or exception thrown
```

3 Ways to Create Threads:

```
// 1. Extend Thread  
class MyThread extends Thread {  
    public void run() { System.out.println("Thread"); }  
}  
new MyThread().start();  
  
// 2. Implement Runnable (preferred)  
Thread t = new Thread(() -> System.out.println("Runnable"));  
t.start();  
  
// 3. Implement Callable (returns result + throws exception)  
Callable<Integer> task = () -> 42;
```

```
Future<Integer> future = executor.submit(task);  
int result = future.get(); // blocks until done
```

Thread Priority:

```
t.setPriority(Thread.MIN_PRIORITY); // 1  
t.setPriority(Thread.NORM_PRIORITY); // 5 (default)  
t.setPriority(Thread.MAX_PRIORITY); // 10  
// WARNING: Priority is just a HINT to OS scheduler  
// Never rely on priority for correctness
```

Daemon Threads:

```
Thread t = new Thread(() -> {  
    while (true) { monitorSystem(); } // background task  
});  
t.setDaemon(true); // JVM exits even if this thread is running  
t.start();  
// Must set BEFORE start() – else IllegalThreadStateException  
// GC thread, JIT compiler are daemon threads
```

2. `wait()`, `notify()`, `notifyAll()` — Monitor Basics

The most fundamental concurrency mechanism — often forgotten:

```
class MessageQueue {  
    private Queue<String> queue = new LinkedList<>();  
    private final int capacity = 10;  
  
    // Producer  
    public synchronized void produce(String msg) throws InterruptedException {  
        while (queue.size() == capacity) {
```

```

        wait(); // releases lock and waits – NOT sleep!
    }
    queue.add(msg);
    notifyAll(); // wake up waiting consumers
}

// Consumer
public synchronized String consume() throws InterruptedException {
    while (queue.isEmpty()) {
        wait(); // releases lock and waits
    }
    String msg = queue.poll();
    notifyAll(); // wake up waiting producers
    return msg;
}
}

```

Critical rules:

- `wait()`, `notify()`, `notifyAll()` MUST be called inside `synchronized` block
- Always use `while` not `if` before `wait()` — handles **spurious wakeups**
- `notify()` wakes ONE random thread, `notifyAll()` wakes ALL waiting threads
- `wait()` **releases** the lock — `sleep()` does NOT release the lock

`sleep()` vs `wait()` — Most Common Interview Question:

	<code>sleep()</code>	<code>wait()</code>
Class	<code>Thread</code>	<code>Object</code>
Releases lock	❌ No	✅ Yes
Where called	Anywhere	Inside <code>synchronized</code>
Woken by	Timer expiry	<code>notify()</code> or timer
Purpose	Pause execution	Inter-thread communication

3. `ThreadLocal` — Deep Dive

```
// Each thread gets its OWN copy of this variable
ThreadLocal<SimpleDateFormat> dateFormat =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

// Thread 1 gets its own SimpleDateFormat instance
// Thread 2 gets its own SimpleDateFormat instance
// No sharing – no synchronization needed!
dateFormat.get().format(new Date()); // thread-safe
```

Real use cases:

- Per-thread DB connections
- Per-request user context in web apps
- SimpleDateFormat (not thread-safe, so use ThreadLocal)

InheritableThreadLocal:

```
// Child threads inherit value from parent thread
InheritableThreadLocal<String> context = new InheritableThreadLocal<>();
context.set("user-123");

new Thread(() -> {
    System.out.println(context.get()); // prints "user-123" – inherited!
}).start();
```

Memory leak — CRITICAL:

```
// In thread pool – ALWAYS call remove() in finally
try {
    threadLocal.set(value);
    doWork();
} finally {
    threadLocal.remove(); // prevents memory leak in pooled threads
}
```

4. **volatile** — Deep Dive

Two guarantees of volatile:

```
volatile boolean flag = false;
volatile long counter = 0;
```

1. **Visibility** — writes immediately visible to all threads
2. **Ordering** — prevents instruction reordering around volatile access

What volatile does NOT guarantee:

```
volatile int count = 0;
count++; // NOT atomic! Read-modify-write is still a race condition
```

Visibility problem without volatile:

```
// WITHOUT volatile – Thread 2 may NEVER see flag = true
// JIT compiler may cache flag in register
boolean flag = false; // NOT volatile

Thread t1 = new Thread(() -> flag = true);
Thread t2 = new Thread(() -> {
    while (!flag) {} // may loop forever!
    System.out.println("Flag seen!");
});
```

Safe publication pattern with volatile:

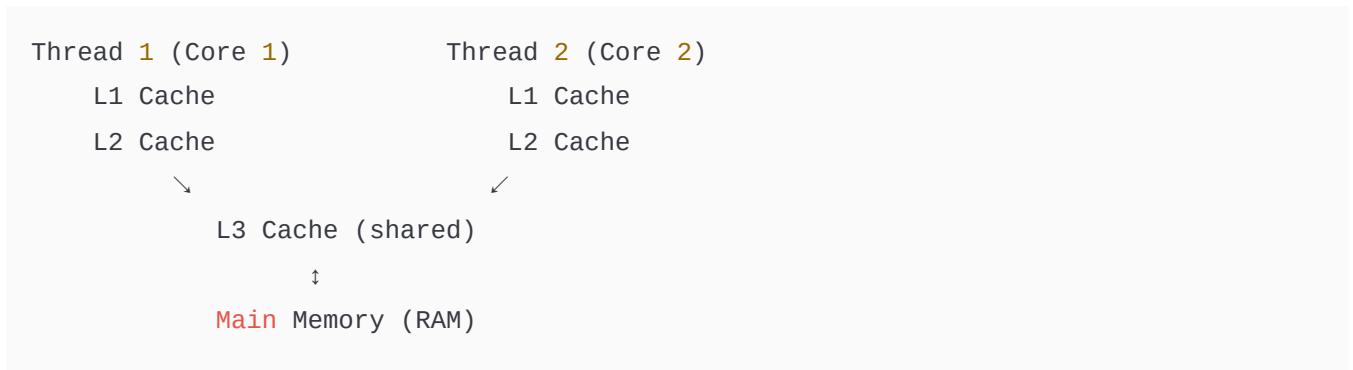
```
volatile Config config = null;

// Writer thread
config = new Config(loadProperties()); // volatile write

// Reader threads – safe, always see fully constructed Config
if (config != null) config.getValue(); // volatile read
```

5. Java Memory Model — Deep Dive

CPU Cache Architecture (why concurrency is hard):



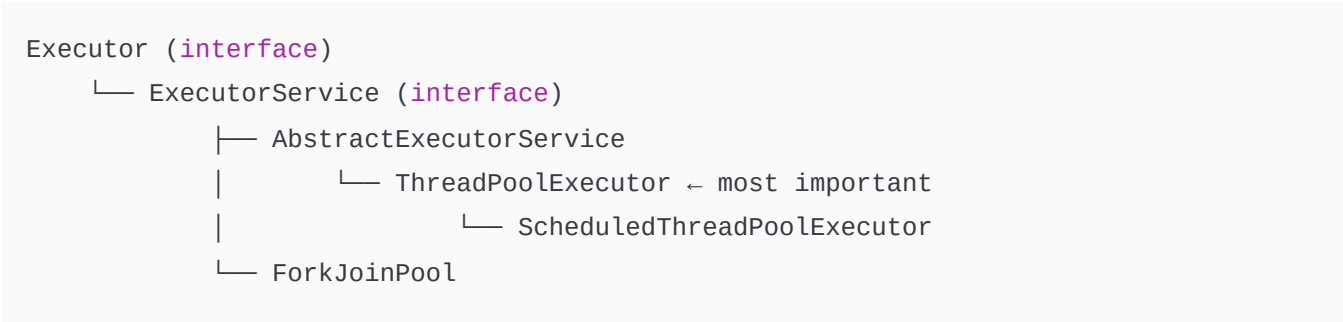
Each core has its own cache. Without synchronization, Thread 2 may read a **stale value** from its cache even after Thread 1 updated main memory.

Happens-Before rules (complete list):

Rule	Description
Program order	Each action in a thread happens-before next action in same thread
Monitor lock	<code>unlock</code> happens-before subsequent <code>lock</code> on same monitor
Volatile	<code>volatile</code> write happens-before subsequent read of same variable
Thread start	<code>thread.start()</code> happens-before any action in started thread
Thread join	All actions in thread happen-before <code>thread.join()</code> returns
Transitivity	If A hb B and B hb C then A hb C

6. Executor Framework — Complete Picture

Full Executor hierarchy:



ScheduledExecutorService — missed commonly:

```
ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);

// Run once after delay
scheduler.schedule(() -> sendAlert(), 5, TimeUnit.SECONDS);

// Run repeatedly – fixed RATE (starts every N seconds regardless of duration)
scheduler.scheduleAtFixedRate(() -> pollPrice(), 0, 1, TimeUnit.SECONDS);

// Run repeatedly – fixed DELAY (waits N seconds AFTER task completes)
scheduler.scheduleWithFixedDelay(() -> cleanCache(), 0, 5, TimeUnit.SECONDS);
```

`scheduleAtFixedRate` vs `scheduleWithFixedDelay`:

```
scheduleAtFixedRate (period=2s):
|--task(3s)--|--task--|--task--| ← overlaps if task > period!

scheduleWithFixedDelay (delay=2s):
|--task(3s)--|..2s..|--task--| ← always waits 2s after completion
```

`Callable` vs `Runnable`:

	<code>Runnable</code>	<code>Callable<V></code>
Return value	✗ void	✓ V
Throws checked exception	✗	✓
Used with	Thread, Executor	ExecutorService
Future result	✗	✓

```
// Runnable
executor.submit(() -> doWork()); // fire and forget

// Callable
Future<Integer> future = executor.submit(() -> {
    return computeResult(); // returns value, can throw
});
int result = future.get(5, TimeUnit.SECONDS); // blocks with timeout
```

7. Future — Complete API

```
Future<String> future = executor.submit(() -> fetchData());

future.get() // block forever
future.get(5, TimeUnit.SECONDS) // block with timeout – throws TimeoutException
future.isDone() // true if completed (normally or exceptionally)
future.isCancelled() // true if cancelled
future.cancel(true) // true = interrupt if running
// false = don't interrupt, just remove from queue
```

Future limitations (why **CompletableFuture** was introduced):

```
// Cannot do any of these with plain Future:
future.thenApply(result -> transform(result)); // no callbacks
future1.combine(future2); // no combining
future.exceptionally(ex -> fallback()); // no exception handling
future.complete("manual"); // no manual completion
```

8. CompletableFuture — Missed Methods

```
// Create already-completed future
CompletableFuture<String> cf = CompletableFuture.completedFuture("done");

// Create failed future
CompletableFuture<String> failed = CompletableFuture.failedFuture(new Exception());

// Run async with no result
CompletableFuture<Void> cf = CompletableFuture.runAsync(() -> doWork());

// thenAccept – consume result, return void
cf.thenAccept(result -> System.out.println(result));

// thenRun – run after completion, no access to result
cf.thenRun(() -> cleanup());
```

```
// orTimeout – fail if not done in time (Java 9+)
cf.orTimeout(5, TimeUnit.SECONDS);

// completeOnTimeout – provide default if timeout (Java 9+)
cf.completeOnTimeout("default", 5, TimeUnit.SECONDS);

// anyOf – first to complete wins
CompletableFuture.anyOf(cf1, cf2, cf3).thenAccept(result -> use(result));
```

Async variants:

```
// thenApply – runs in same thread or completing thread
// thenApplyAsync – runs in ForkJoinPool.commonPool()
// thenApplyAsync(fn, executor) – runs in YOUR executor (preferred)

cf.thenApply(this::transform) // same thread
cf.thenApplyAsync(this::transform) // common pool
cf.thenApplyAsync(this::transform, myPool) // your pool – safest
```

9. Concurrent Collections — Complete List

```
// Thread-safe Map
ConcurrentHashMap<K,V> // best general purpose
ConcurrentSkipListMap<K,V> // sorted, thread-safe (like TreeMap)

// Thread-safe Set
ConcurrentSkipListSet<E> // sorted thread-safe set
CopyOnWriteArraySet<E> // read-heavy small sets

// Thread-safe List
CopyOnWriteArrayList<E> // read-heavy lists

// Thread-safe Queues
ArrayBlockingQueue<E> // bounded, blocking
LinkedBlockingQueue<E> // optionally bounded, blocking
PriorityBlockingQueue<E> // priority ordered, blocking
DelayQueue<E> // elements available after delay
SynchronousQueue<E> // zero capacity, direct handoff
```

```
LinkedTransferQueue<E>           // producer waits for consumer
ConcurrentLinkedQueue<E>         // lock-free, non-blocking
ConcurrentLinkedDeque<E>         // lock-free double-ended queue
```

DelayQueue — often missed:

```
// Elements only become available after their delay expires
DelayQueue<ScheduledTask> queue = new DelayQueue<>();

queue.put(new ScheduledTask("task1", 5, SECONDS)); // available after 5s
queue.put(new ScheduledTask("task2", 2, SECONDS)); // available after 2s

ScheduledTask task = queue.take(); // blocks until earliest delay expires
// Returns task2 first (2s < 5s)
```

Used for: retry queues, scheduled task execution, session expiry.

10. Atomic Classes — Complete List

```
// Basic atomics
AtomicBoolean  flag    = new AtomicBoolean(false);
AtomicInteger  count    = new AtomicInteger(0);
AtomicLong     id       = new AtomicLong(0);
AtomicReference<T> ref  = new AtomicReference<>(initial);

// Array atomics
AtomicIntegerArray arr = new AtomicIntegerArray(10);
AtomicLongArray    arr = new AtomicLongArray(10);
AtomicReferenceArray<T> arr = new AtomicReferenceArray<>(10);

// Field updaters – update volatile fields atomically without wrapping
AtomicIntegerFieldUpdater<Node> updater =
    AtomicIntegerFieldUpdater.newUpdater(Node.class, "value");
updater.compareAndSet(node, expected, update);

// High-contention counters (Java 8)
LongAdder      adder = new LongAdder(); // faster than AtomicLong under contention
LongAccumulator acc = new LongAccumulator(Long::max, 0); // custom accumulation
DoubleAdder    dadder = new DoubleAdder();
```

```
// ABA-safe
AtomicStampedReference<V> // version stamp
AtomicMarkableReference<V> // boolean mark (e.g., logically deleted)
```

LongAdder vs **AtomicLong** :

AtomicLong: all threads CAS on ONE variable → contention under load
LongAdder: each thread updates its OWN Cell → sum() adds all cells
much faster for increment-heavy scenarios (metrics, counters)
trade-off: sum() is not a precise atomic snapshot

11. Fork/Join Framework — Deep Dive

```
// RecursiveTask – returns a result
class SumTask extends RecursiveTask<Long> {
    private final int[] arr;
    private final int start, end;
    private static final int THRESHOLD = 1000;

    protected Long compute() {
        if (end - start <= THRESHOLD) {
            // Base case – compute directly
            long sum = 0;
            for (int i = start; i < end; i++) sum += arr[i];
            return sum;
        }
        // Divide
        int mid = (start + end) / 2;
        SumTask left = new SumTask(arr, start, mid);
        SumTask right = new SumTask(arr, mid, end);

        left.fork(); // submit left asynchronously
        long rightResult = right.compute(); // compute right in current thread
        long leftResult = left.join(); // wait for left

        return leftResult + rightResult;
    }
}
```

```

}

ForkJoinPool pool = new ForkJoinPool();
long total = pool.invoke(new SumTask(arr, 0, arr.length));

```

RecursiveAction — no return value:

```

class SortTask extends RecursiveAction {
    protected void compute() {
        if (size <= THRESHOLD) {
            Arrays.sort(arr, start, end);
            return;
        }
        SortTask left = new SortTask(arr, start, mid);
        SortTask right = new SortTask(arr, mid, end);
        invokeAll(left, right); // fork both and wait
        merge(arr, start, mid, end);
    }
}

```

12. Thread Interruption — Complete Picture

```

// Interruption is COOPERATIVE – you must check for it
Thread t = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) { // check flag
        doWork();
    }
    cleanup(); // graceful exit
});

t.interrupt(); // sets interrupt flag = true

// Two ways interrupt flag is consumed:
// 1. Thread.isInterrupted() – checks flag, does NOT clear it
// 2. Thread.interrupted() – checks flag AND clears it (static method)

// Blocking methods that throw InterruptedException:
// Thread.sleep(), Object.wait(), BlockingQueue.take(),

```

```
// Lock.lockInterruptibly(), Future.get() etc.  
// These CLEAR the interrupt flag when they throw!  
  
// So ALWAYS restore it:  
try {  
    Thread.sleep(1000);  
} catch (InterruptedException e) {  
    Thread.currentThread().interrupt(); // restore flag!  
    return; // exit cleanly  
}
```

13. Object-Level vs Class-Level Locking

```
public class Counter {  
  
    // Object-level lock – locks THIS instance only  
    public synchronized void increment() { count++; }  
  
    // Equivalent to:  
    public void increment() {  
        synchronized (this) { count++; }  
    }  
  
    // Class-level lock – locks the CLASS across ALL instances  
    public static synchronized void reset() { staticCount = 0; }  
  
    // Equivalent to:  
    public static void reset() {  
        synchronized (Counter.class) { staticCount = 0; }  
    }  
}
```

Key: Two different `Counter` instances can execute `increment()` simultaneously — they use **different object locks**. But only one thread can execute `reset()` at a time — it locks the **class object**.

14. Exchanger — Rarely Known

```
// Two threads swap data at a synchronization point
Exchanger<List<Order>> exchanger = new Exchanger<>();

// Thread 1 – fills buffer and exchanges
Thread producer = new Thread(() -> {
    List<Order> buffer = fillBuffer();
    List<Order> emptyBuffer = exchanger.exchange(buffer); // swap!
    // now has empty buffer to refill
});

// Thread 2 – drains buffer and exchanges
Thread consumer = new Thread(() -> {
    List<Order> emptyBuffer = new ArrayList<>();
    List<Order> fullBuffer = exchanger.exchange(emptyBuffer); // swap!
    // now has full buffer to drain
    process(fullBuffer);
});
```

Used for **double-buffering** — producer fills one buffer while consumer drains the other.

15. Common Concurrency Anti-Patterns

1. Calling `Thread.stop()` — deprecated and dangerous:

```
t.stop(); // NEVER use – leaves objects in inconsistent state
t.suspend(); // NEVER use – prone to deadlock
t.resume(); // NEVER use
// Use interrupt() and cooperative cancellation instead
```

2. Synchronizing on String literals — dangerous:

```
synchronized ("USER_LOCK") { } // WRONG – string literals are interned
                                // all code using "USER_LOCK" shares same lock!
synchronized (new String("lock")) { } // WRONG – new object each time = no sync
// Use: private final Object lock = new Object();
```

3. Returning mutable objects from synchronized methods:

```
private List<Order> orders = new ArrayList<>();

public synchronized List<Order> getOrders() {
    return orders; // WRONG – caller gets reference, can modify unsafely
}

public synchronized List<Order> getOrders() {
    return Collections.unmodifiableList(new ArrayList<>(orders)); // CORRECT
}
```

4. Using `volatile` for compound actions:

```
volatile int count = 0;
count++;           // WRONG – not atomic even with volatile
// Use AtomicInteger
```

5. Ignoring `InterruptedException`:

```
catch (InterruptedException e) { } // WRONG – swallows interrupt signal
```

6. Over-synchronization:

```
public synchronized int getSize() { return list.size(); } // unnecessary if read-only
public synchronized String toString() { return "..."; } // locks for logging!
```

16. Virtual Threads (Java 21) — Modern Concurrency

```
// Traditional thread – expensive, maps to OS thread
Thread t = new Thread(() -> doWork());

// Virtual thread – lightweight, managed by JVM (Java 21)
Thread vt = Thread.ofVirtual().start(() -> doWork());

// Virtual thread executor
```

```

ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor();

// Create millions of virtual threads – unlike platform threads
IntStream.range(0, 1_000_000).forEach(i ->
    Thread.ofVirtual().start(() -> handleRequest(i))
);

```

Key differences:

	Platform Thread	Virtual Thread
Maps to	OS thread	JVM-managed
Cost	~1MB stack	~few KB
Max practical	~thousands	Millions
Blocking I/O	Blocks OS thread	Unmounts, frees carrier thread
Best for	CPU-bound	I/O-bound (HTTP, DB calls)

Virtual threads automatically unmount from carrier thread during blocking operations — the OS thread is freed to run other virtual threads. This makes blocking code scale like async code without callback hell.

🔴 17. Structured Concurrency (Java 21) — Brand New

```

// All subtasks treated as a unit – if one fails, others are cancelled
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    Future<User> user = scope.fork(() -> fetchUser(id));
    Future<Order> order = scope.fork(() -> fetchOrder(id));

    scope.join();           // wait for both
    scope.throwIfFailed();  // propagate any exception

    return new Response(user.resultNow(), order.resultNow());
}
// Scope closes here – all tasks guaranteed complete or cancelled

```

Complete Topic Checklist

Category	Topics
Thread Basics	Lifecycle, creation, priority, daemon threads
Monitor	wait/notify/notifyAll, sleep vs wait
Visibility	volatile, JMM, happens-before, CPU cache
Locks	synchronized, ReentrantLock, ReadWriteLock, StampedLock
Atomic	AtomicInteger, LongAdder, CAS, ABA, field updaters
Collections	ConcurrentHashMap, CopyOnWriteArrayList, BlockingQueues, DelayQueue
Coordination	CountDownLatch, CyclicBarrier, Phaser, Semaphore, Exchanger
Executors	ThreadPoolExecutor, ScheduledExecutor, ForkJoinPool
Async	Future, CompletableFuture, all methods
ThreadLocal	Usage, InheritableThreadLocal, memory leaks
Interruption	interrupt(), isInterrupted(), InterruptedException handling
Patterns	Producer-consumer, pipeline, double-checked locking, holder
Anti-patterns	stop/suspend, sync on String, swallow interrupt, over-sync
Modern	Virtual threads, structured concurrency (Java 21)
Debugging	Thread dumps, deadlock detection, JFR, async-profiler

4/23/2026, 8:34:13 AM

 java8 addition to multithreading or concurrency added here ?

4/23/2026, 8:35:09 AM

Here are **all Java 8 additions to Multithreading & Concurrency** specifically:

1. **CompletableFuture** — Biggest Addition

Completely new in Java 8. Solved all **Future** limitations.

Creation

```
// Run async – no result
CompletableFuture<Void> cf = CompletableFuture.runAsync(() -> doWork());

// Supply async – with result
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() -> fetchData());

// Supply async – with YOUR executor (always preferred)
CompletableFuture<String> cf = CompletableFuture.supplyAsync(
    () -> fetchData(), myExecutor
);

// Already completed
CompletableFuture<String> cf = CompletableFuture.completedFuture("done");
```

Transformation Chain

```
CompletableFuture
    .supplyAsync(() -> fetchUserId())           // returns 101
    .thenApply(id -> fetchUser(id))             // transform: id → User
    .thenApply(user -> user.getName())          // transform: User → String
    .thenAccept(name -> System.out.println(name)) // consume, no return
    .thenRun(() -> log.info("Pipeline done")); // run after, no input/output
```

Chaining Async Steps

```
// thenCompose – flatMap for CompletableFuture
// Use when next step is ALSO async

CompletableFuture<Order> result =
    CompletableFuture.supplyAsync(() -> getUserId())           // CF<Integer>
        .thenCompose(id -> fetchOrderAsync(id))                // CF<Order> – unwrapped!
        .thenCompose(order -> enrichOrderAsync(order));        // CF<Order>

// thenApply vs thenCompose
// thenApply(id -> fetchOrderAsync(id)) → CF<CF<Order>>  WRONG
// thenCompose(id -> fetchOrderAsync(id)) → CF<Order>    CORRECT
```

Combining Two Independent Futures

```
CompletableFuture<String> userFuture = fetchUserAsync();
CompletableFuture<String> orderFuture = fetchOrderAsync();

// Wait for BOTH – combine results
userFuture.thenCombine(orderFuture, (user, order) -> user + " bought " + order)
    .thenAccept(System.out::println);

// Wait for BOTH – no result needed
CompletableFuture.allOf(userFuture, orderFuture).thenRun(() -> {
    String user = userFuture.join();
    String order = orderFuture.join();
    merge(user, order);
});

// First one to complete wins
CompletableFuture.anyOf(cf1, cf2, cf3)
    .thenAccept(result -> handleFirst(result));
```

Exception Handling

```
CompletableFuture
    .supplyAsync(() -> riskyOperation())
```

```

// Handle ONLY failure – provide fallback
.exceptionally(ex -> {
    log.error("Failed", ex);
    return "fallback";
})

// Handle BOTH success and failure – can transform
.handle((result, ex) -> {
    if (ex != null) return "error: " + ex.getMessage();
    return "success: " + result;
})

// Side-effect on BOTH – result still propagates unchanged
.whenComplete((result, ex) -> {
    if (ex != null) metrics.incrementError();
    else metrics.incrementSuccess();
});

```

Manual Completion

```

CompletableFuture<String> cf = new CompletableFuture<>();

// From another thread – complete it manually
executor.submit(() -> {
    try {
        String result = doHeavyWork();
        cf.complete(result); // completes normally
    } catch (Exception e) {
        cf.completeExceptionally(e); // completes exceptionally
    }
});

// Check state
cf.isDone();
cf.isCompletedExceptionally();
cf.isCancelled();

// Force complete regardless of current state
cf.obtrudeValue("forced");
cf.obtrudeException(new RuntimeException());

```

Async Variants — Important Difference

```
// Sync – runs in same thread that completed the previous stage
.thenApply(fn)

// Async – runs in ForkJoinPool.commonPool()
.thenApplyAsync(fn)

// Async – runs in YOUR executor (ALWAYS prefer this for I/O)
.thenApplyAsync(fn, myExecutor)

// Same pattern applies to:
// thenAccept / thenAcceptAsync / thenAcceptAsync(fn, executor)
// thenRun / thenRunAsync / thenRunAsync(fn, executor)
// thenCompose / thenComposeAsync
// thenCombine / thenCombineAsync
```

2. LongAdder and LongAccumulator

LongAdder — High-Contention Counter

```
// AtomicLong – all threads fight over ONE variable
AtomicLong atomicLong = new AtomicLong();
atomicLong.incrementAndGet(); // CAS retry under contention

// LongAdder – each thread updates its OWN Cell
LongAdder adder = new LongAdder();
adder.increment(); // updates thread-local cell – no contention
adder.add(5);
long total = adder.sum(); // sums all cells – eventual consistency
adder.reset(); // resets to 0
long sumAndReset = adder.sumThenReset();
```

Internally:

LongAdder internals:

Low contention:

Thread 1 → base variable (shared)

High contention:

Thread 1 → Cell[0]

Thread 2 → Cell[1]

Thread 3 → Cell[2]

Thread 4 → Cell[3]

sum() = base + Cell[0] + Cell[1] + Cell[2] + Cell[3]

When to use which:

AtomicLong → need exact value at a point (e.g., generate unique ID)

LongAdder → pure counting/summing under high contention (e.g., request counter)

LongAccumulator — Custom Accumulation

```
// LongAccumulator(operation, identity)
LongAccumulator maxVal = new LongAccumulator(Long::max, Long.MIN_VALUE);
LongAccumulator minVal = new LongAccumulator(Long::min, Long.MAX_VALUE);
LongAccumulator product = new LongAccumulator((a, b) -> a * b, 1L);

// Multiple threads accumulate
maxVal.accumulate(150); // Thread 1
maxVal.accumulate(200); // Thread 2
maxVal.accumulate(180); // Thread 3

long max = maxVal.get(); // 200
```

Also added: DoubleAdder, DoubleAccumulator — same concept for doubles.

3. ForkJoinPool.commonPool() — Exposed in Java 8

```
// Java 8 exposed the common pool publicly
ForkJoinPool pool = ForkJoinPool.commonPool();
```

```
// Size = Runtime.getRuntime().availableProcessors() - 1
System.out.println(pool.getParallelism()); // e.g., 7 on 8-core machine

// Used AUTOMATICALLY by:
// 1. CompletableFuture.supplyAsync() – without custom executor
// 2. Parallel streams
// 3. Arrays.parallelSort()
```

Risk — shared pool starvation:

```
// ALL of these share the SAME common pool
list.parallelStream().map(this::slowIO).collect(toList()); // blocks pool threads
CompletableFuture.supplyAsync(() -> callAPI());             // also uses common pool
Arrays.parallelSort(bigArray);                               // also uses common pool

// Fix – use custom pool for I/O bound tasks
ForkJoinPool customPool = new ForkJoinPool(20);
customPool.submit(() ->
    list.parallelStream().map(this::slowIO).collect(toList())
).get();
```

4. Parallel Streams — Java 8

```
List<Integer> numbers = List.of(1,2,3,4,5,6,7,8,9,10);

// Sequential stream
numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .collect(Collectors.toList());

// Parallel stream – uses ForkJoinPool.commonPool()
numbers.parallelStream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .collect(Collectors.toList());
```

```
// Convert existing stream to parallel
numbers.stream()
    .parallel()
    .map(this::heavyComputation)
    .collect(Collectors.toList());
```

Thread Safety in Parallel Streams

```
// WRONG – ArrayList is not thread-safe
List<Integer> result = new ArrayList<>();
numbers.parallelStream()
    .forEach(n -> result.add(n)); // race condition!

// CORRECT – use collect()
List<Integer> result = numbers.parallelStream()
    .filter(n -> n > 5)
    .collect(Collectors.toList()); // thread-safe collector

// CORRECT – use thread-safe collection
List<Integer> result = new CopyOnWriteArrayList<>();
numbers.parallelStream().forEach(result::add);

// CORRECT – use atomic for aggregation
AtomicInteger sum = new AtomicInteger();
numbers.parallelStream().forEach(sum::addAndGet);
```

When Parallel Streams Help vs Hurt

```
// HELPS – large data + CPU-bound
largeList.parallelStream() // 1M+ elements
    .map(this::cpuHeavyWork) // pure computation, no I/O
    .collect(toList());

// HURTS – small data (thread overhead > benefit)
smallList.parallelStream() // 10 elements
    .map(n -> n * 2) // trivial operation
    .collect(toList()); // slower than sequential!
```

```
// HURTS – I/O bound (blocks common pool threads)
urls.parallelStream()
    .map(this::httpCall)           // blocks threads
    .collect(toList());           // use CompletableFuture instead

// HURTS – ordered operations
numbers.parallelStream()
    .forEachOrdered(System.out::println); // forces ordering = kills parallelism
```

5. **StampedLock** — Added in Java 8

```
StampedLock lock = new StampedLock();

// WRITE LOCK – exclusive
long stamp = lock.writeLock();
try {
    updateData();
} finally {
    lock.unlockWrite(stamp);
}

// READ LOCK – shared (like ReadWriteLock)
long stamp = lock.readLock();
try {
    return readData();
} finally {
    lock.unlockRead(stamp);
}

// OPTIMISTIC READ – no lock at all!
long stamp = lock.tryOptimisticRead();
int data = this.data;           // read WITHOUT lock
if (!lock.validate(stamp)) {    // check if write happened during read
    stamp = lock.readLock();     // fall back to real read lock
    try {
        data = this.data;
    } finally {
        lock.unlockRead(stamp);
    }
}
```

```
}  
return data;
```

Lock conversion:

```
// Upgrade read lock to write lock  
long stamp = lock.readLock();  
try {  
    if (needsUpdate()) {  
        long writeStamp = lock.tryConvertToWriteLock(stamp);  
        if (writeStamp != 0) {  
            stamp = writeStamp;  
            updateData();  
        } else {  
            lock.unlockRead(stamp);  
            stamp = lock.writeLock(); // acquire write lock fresh  
            updateData();  
        }  
    }  
} finally {  
    lock.unlock(stamp);  
}
```

StampedLock limitations:

- ❌ NOT reentrant – same thread cannot acquire twice
- ❌ No Condition support
- ❌ No interruptible lock acquisition (use tryLock variants)
- ✅ Best performance for read-heavy workloads

6. ConcurrentHashMap New Methods — Java 8

Atomic Compute Methods

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();  
  
// computeIfAbsent – lazy init, atomic  
map.computeIfAbsent("AAPL", k -> new ArrayList<>()); // only creates if absent
```

```

// computeIfPresent – update only if key exists
map.computeIfPresent("AAPL", (k, v) -> v + 1);

// compute – always update (insert or update)
map.compute("AAPL", (k, v) -> v == null ? 1 : v + 1);

// merge – cleanest way to accumulate
map.merge("AAPL", 1, Integer::sum); // if absent: put 1, if present: add 1

// putIfAbsent – returns OLD value if present, null if inserted
Integer old = map.putIfAbsent("AAPL", 100);

// getOrDefault
int val = map.getOrDefault("AAPL", 0);

// replaceAll – update all values atomically
map.replaceAll((k, v) -> v * 2);

```

Bulk Operations — New in Java 8

```

ConcurrentHashMap<String, Integer> prices = new ConcurrentHashMap<>();

// forEach – parallel iteration
prices.forEach(2, // parallelism threshold
    (symbol, price) -> System.out.println(symbol + "=" + price)
);

// search – returns first non-null result
String found = prices.search(2,
    (k, v) -> v > 1000 ? k : null // find first symbol with price > 1000
);

// reduce – parallel aggregation
int total = prices.reduce(2,
    (k, v) -> v, // transformer: value per entry
    Integer::sum // reducer: combine values
);

// mappingCount – better than size() for large maps
long count = prices.mappingCount(); // returns long, not int

```

7. `Arrays.parallelSort()` — Java 8

```
int[] arr = {5, 3, 8, 1, 9, 2, 7};

// Sequential sort – O(n log n), single thread
Arrays.sort(arr);

// Parallel sort – uses ForkJoinPool, faster for large arrays
Arrays.parallelSort(arr);

// Partial range
Arrays.parallelSort(arr, 2, 6); // sort indices 2 to 5 only

// With comparator (for objects)
String[] names = {"Charlie", "Alice", "Bob"};
Arrays.parallelSort(names, String::compareTo);
```

When parallel sort wins:

```
Small arrays (<~4000 elements) → Arrays.sort() faster (no thread overhead)
Large arrays (>~4000 elements) → Arrays.parallelSort() faster
```

8. `Arrays.parallelPrefix()` and `Arrays.parallelSetAll()`

```
int[] arr = {1, 2, 3, 4, 5};

// parallelSetAll – initialize array in parallel
Arrays.parallelSetAll(arr, i -> i * i);
// arr = [0, 1, 4, 9, 16]

// parallelPrefix – cumulative operation in parallel
Arrays.parallelPrefix(arr, Integer::sum);
// arr = [0, 1, 5, 14, 30] ← running sum
// Equivalent to: arr[i] = arr[0] + arr[1] + ... + arr[i]
```

9. Lambda & Method References in Concurrency — Java 8

```
// Before Java 8 – anonymous class (verbose)
ExecutorService executor = Executors.newFixedThreadPool(5);
executor.submit(new Runnable() {
    public void run() { doWork(); }
});

// Java 8 – lambda (clean)
executor.submit(() -> doWork());

// Method reference
executor.submit(this::doWork);

// Comparator with lambda in concurrent collections
ConcurrentSkipListMap<String, Integer> map =
    new ConcurrentSkipListMap<>((a, b) -> b.compareTo(a)); // reverse order

// Cleaner thread creation
Thread t = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        processNextItem();
    }
});
```

🎯 Java 8 Concurrency Additions — Complete Summary

Feature	What it Adds
CompletableFuture	Async pipelines, callbacks, combining futures
LongAdder	High-contention counter — faster than AtomicLong
LongAccumulator	Custom parallel accumulation
DoubleAdder	Same as LongAdder for doubles
DoubleAccumulator	Same as LongAccumulator for doubles
StampedLock	Optimistic reads, better than ReadWriteLock

Feature	What it Adds
<code>ForkJoinPool.commonPool()</code>	Exposed shared pool, used by parallel streams
Parallel Streams	Easy data parallelism with <code>.parallelStream()</code>
<code>ConcurrentHashMap.compute()</code>	Atomic compute, merge, forEach, reduce, search
<code>ConcurrentHashMap.mappingCount()</code>	Returns long count
<code>Arrays.parallelSort()</code>	Parallel merge sort
<code>Arrays.parallelSetAll()</code>	Parallel array initialization
<code>Arrays.parallelPrefix()</code>	Parallel cumulative operations
Lambdas in concurrency	Cleaner Runnable, Callable, thread creation

4/23/2026, 8:36:15 AM

 all multithreading and concurrency question answwr covered here or pending

4/23/2026, 8:36:47 AM

Let me do a **complete audit** of everything covered and what's still pending:

COVERED TOPICS

#	Topic	Where Covered
1	synchronized, volatile, Atomic	Q&A Session 1
2	ReentrantLock vs synchronized	Q&A + VS Comparison
3	ReadWriteLock vs StampedLock	Q&A + VS Comparison
4	CountDownLatch, CyclicBarrier, Phaser, Semaphore	Q&A + Explanation
5	CompletableFuture full API	Java 8 Session
6	LongAdder, LongAccumulator	Java 8 Session
7	ForkJoinPool, Work Stealing	JPMC + Java 8
8	Parallel Streams	Java 8 Session
9	StampedLock	Java 8 Session
10	ConcurrentHashMap internals + new methods	All Sessions
11	ThreadPoolExecutor parameters	Q&A Session 1
12	Thread pool sizing	MBB Labs
13	Deadlock, Livelock, Starvation	Q&A + VS
14	Race condition vs Data race	Q&A + VS
15	ABA problem, AtomicStampedReference	Q&A Session 1
16	False sharing, @Contended	Q&A Session 1
17	ThreadLocal + memory leak	JPMC + Missed Topics
18	JMM, Happens-Before	Q&A + JPMC
19	Thread lifecycle states	Missed Topics
20	wait/notify/notifyAll	Missed Topics
21	sleep vs wait	Missed Topics
22	Daemon threads	Missed Topics

#	Topic	Where Covered
23	Thread interruption	Missed Topics
24	LockSupport.park/unpark	Q&A Session 1
25	AQS internals	JPMC
26	Producer-consumer pattern	All Sessions
27	Cache stampede	MBB Labs
28	Order pipeline pattern	MBB Labs
29	Lazy init / Double checked locking	Q&A + MBB
30	Graceful shutdown	MBB Labs
31	Concurrent collections list	Missed Topics
32	DelayQueue	Missed Topics
33	Exchanger	Missed Topics
34	Arrays.parallelSort/prefix/setAll	Java 8 Session
35	Virtual Threads (Java 21)	Missed Topics
36	Structured Concurrency (Java 21)	Missed Topics
37	Anti-patterns	Missed Topics
38	Debugging — thread dump, JFR	Q&A Session 1
39	VS comparisons (15 comparisons)	VS Session

✗ PENDING TOPICS — Not Yet Covered

🔴 PENDING 1: synchronized Deep Dive — Biased/Thin/Fat Lock

Not covered — JVM internally has 3 lock types:

Biased Lock	→ only one thread ever accesses → no CAS needed
Thin Lock	→ low contention → CAS-based spin lock
Fat Lock	→ high contention → OS mutex, thread sleeps

JVM automatically escalates:

Biased → Thin → Fat (based on contention level)

🔴 PENDING 2: `ReentrantReadWriteLock` — Complete API

Only compared, never fully explained:

```
ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();

// Multiple readers simultaneously
readLock.lock();
try { return data; }
finally { readLock.unlock(); }

// Exclusive writer
writeLock.lock();
try { data = newValue; }
finally { writeLock.unlock(); }

// Lock downgrade – write → read (allowed)
writeLock.lock();
try {
    update();
    readLock.lock(); // acquire read before releasing write
} finally {
    writeLock.unlock(); // downgrade complete
}

// Lock upgrade – read → write (NOT allowed – causes deadlock)
```

🔴 PENDING 3: `Phaser` — Deep Dive

Mentioned but never fully explained:

```

Phaser phaser = new Phaser(3); // 3 parties

// Each thread does multiple phases
Runnable task = () -> {
    doPhase1();
    phaser.arriveAndAwaitAdvance(); // wait for all at phase 1

    doPhase2();
    phaser.arriveAndAwaitAdvance(); // wait for all at phase 2

    doPhase3();
    phaser.arriveAndDeregister(); // done – deregister from phaser
};

// Dynamic registration
phaser.register(); // add a new party at runtime
phaser.bulkRegister(5); // add 5 parties at once

// Monitor phase number
int phase = phaser.getPhase();

// Termination
phaser.forceTermination();
phaser.isTerminated();

```

PENDING 4: **BlockingDeque** — Never Covered

```

// Double-ended blocking queue – work stealing basis
BlockingDeque<Task> deque = new LinkedBlockingDeque<>(100);

// Add/remove from FRONT
deque.putFirst(task);
Task t = deque.takeFirst();

// Add/remove from END
deque.putLast(task);
Task t = deque.takeLast();

// Non-blocking variants

```

```
deque.offerFirst(task);
deque.pollLast();
```

```
// Use case – work stealing
// Worker threads push/pop from their own deque head
// Idle threads steal from other deques' tail
```

PENDING 5: `CyclicBarrier` — Deep Dive

Only compared, never fully shown:

```
// With barrier action – runs when all threads arrive
CyclicBarrier barrier = new CyclicBarrier(3, () -> {
    System.out.println("All arrived! Starting next phase...");
    mergeResults(); // runs in last arriving thread
});

// Thread usage
barrier.await(); // wait indefinitely
barrier.await(5, TimeUnit.SECONDS); // wait with timeout
barrier.getNumberWaiting(); // how many waiting now
barrier.getParties(); // total parties
barrier.isBroken(); // true if broken by timeout/interrupt
barrier.reset(); // reset to initial state – waiting threads get
BrokenBarrierException

// BrokenBarrierException – when barrier is broken
try {
    barrier.await();
} catch (BrokenBarrierException e) {
    // another thread timed out or was interrupted
    // or reset() was called
}
```

PENDING 6: Lock Conditions — `newCondition()`

Never covered — very important:

```
ReentrantLock lock = new ReentrantLock();
Condition notFull = lock.newCondition(); // wait when queue full
Condition notEmpty = lock.newCondition(); // wait when queue empty

// Producer
lock.lock();
try {
    while (queue.size() == capacity)
        notFull.await(); // releases lock and waits
    queue.add(item);
    notEmpty.signal(); // wake ONE waiting consumer
} finally { lock.unlock(); }

// Consumer
lock.lock();
try {
    while (queue.isEmpty())
        notEmpty.await(); // releases lock and waits
    Item item = queue.poll();
    notFull.signal(); // wake ONE waiting producer
    return item;
} finally { lock.unlock(); }
```

vs **Object.wait/notify**:

```
Object.wait() → ONE condition queue per object
Condition     → MULTIPLE condition queues per lock
               (notFull AND notEmpty separately)
               More precise – no spurious wakeups of wrong waiters
```

PENDING 7: **ThreadFactory** — Never Covered

```
// Custom ThreadFactory – control thread naming, priority, daemon status
ThreadFactory factory = new ThreadFactory() {
    AtomicInteger count = new AtomicInteger(0);
```

```

public Thread newThread(Runnable r) {
    Thread t = new Thread(r);
    t.setName("order-processor-" + count.incrementAndGet());
    t.setDaemon(true);
    t.setPriority(Thread.NORM_PRIORITY);
    t.setUncaughtExceptionHandler((thread, ex) ->
        log.error("Uncaught in {}: {}", thread.getName(), ex)
    );
    return t;
}
};

ExecutorService pool = Executors.newFixedThreadPool(10, factory);

// Or use Guava's ThreadFactoryBuilder
ThreadFactory factory = new ThreadFactoryBuilder()
    .setNameFormat("worker-%d")
    .setDaemon(true)
    .build();

```

PENDING 8: `UncaughtExceptionHandler` — Never Covered

```

// Without handler – exception silently swallowed in thread pools!
executor.submit(() -> {
    throw new RuntimeException("BOOM"); // nobody sees this!
});

// Per-thread handler
Thread t = new Thread(() -> riskyWork());
t.setUncaughtExceptionHandler((thread, ex) -> {
    log.error("Thread {} threw: {}", thread.getName(), ex.getMessage());
    alertOpsTeam(ex);
});

// Global default handler – catches ALL unhandled exceptions
Thread.setDefaultUncaughtExceptionHandler((thread, ex) -> {
    log.error("UNHANDLED in {}", thread.getName(), ex);
});

```

```
// In thread pool – use try-catch inside task OR override afterExecute
class MonitoredThreadPool extends ThreadPoolExecutor {
    protected void afterExecute(Runnable r, Throwable t) {
        super.afterExecute(r, t);
        if (t == null && r instanceof Future<?>) {
            try { ((Future<?>)r).get(); }
            catch (ExecutionException ee) { t = ee.getCause(); }
            catch (InterruptedException ie) { Thread.currentThread().interrupt(); }
        }
        if (t != null) log.error("Task failed: ", t);
    }
}
```

PENDING 9: **TransferQueue** — Never Covered

```
TransferQueue<String> queue = new LinkedTransferQueue<>();

// Producer – two modes
queue.put("msg");           // add to queue, don't wait for consumer
queue.transfer("msg");      // BLOCK until a consumer takes it directly

// tryTransfer – non-blocking
boolean taken = queue.tryTransfer("msg");           // returns false if no consumer
boolean taken = queue.tryTransfer("msg", 1, SECONDS); // wait up to 1 second

// Consumer
String msg = queue.take();

// Check waiting consumers
int waiting = queue.getWaitingConsumerCount();
boolean hasConsumer = queue.hasWaitingConsumer();
```

Use case: When producer needs confirmation that consumer **received** the message before continuing — like a synchronous handoff.

PENDING 10: Memory Consistency — **final** fields

Never covered — critical for safe publication:

```
// final fields are SAFELY PUBLISHED without synchronization
class ImmutablePoint {
    final int x;
    final int y;

    ImmutablePoint(int x, int y) {
        this.x = x; // JMM guarantees: after constructor,
        this.y = y; // ALL threads see correct x and y values
    }
}

// Shared without synchronization – SAFE because fields are final
ImmutablePoint point = new ImmutablePoint(3, 4);
// All threads see x=3, y=4 – guaranteed by JMM final field semantics

// Non-final – NOT safe without volatile/synchronized
class MutablePoint {
    int x; // NOT final
    int y; // NOT final
    // Another thread might see x=3, y=0 (partial construction!)
}
```

PENDING 11: `CopyOnWriteArrayList` — Internal Working

Only compared, never explained internally:

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();

// WRITE – creates entire new array copy
list.add("item"); // copies array, adds, replaces reference
list.remove("item"); // copies array, removes, replaces reference
// Lock held during copy – writes are mutually exclusive

// READ – always reads from current snapshot, no lock
list.get(0); // lock-free!
list.size(); // lock-free!
```

```
// ITERATION – snapshot at time of iterator creation
Iterator<String> it = list.iterator(); // takes snapshot
list.add("new"); // modifies list – iterator UNAFFECTED
// it still iterates old snapshot – no ConcurrentModificationException

// When to use
// ✅ Event listener lists (rare add/remove, frequent iteration)
// ✅ Config/whitelist (updated rarely, read constantly)
// ❌ Large lists with frequent writes – O(n) copy every write
// ❌ Lists needing up-to-date reads during iteration
```

🔴 PENDING 12: Happens-Before — Practical Pitfalls

```
// PITFALL 1 – Double publication without volatile
String value = null;
boolean ready = false; // NOT volatile

Thread writer = new Thread(() -> {
    value = "hello"; // write 1
    ready = true;    // write 2
    // JIT can REORDER these – ready=true before value="hello"!
});

Thread reader = new Thread(() -> {
    if (ready) { // sees ready=true
        print(value); // may still see null! No happens-before!
    }
});

// FIX – make ready volatile
volatile boolean ready = false; // volatile write to ready
// happens-after write to value
```

```
// PITFALL 2 – Safe publication via static initializer
class Singleton {
    // JVM guarantees class initialization is thread-safe
    static final Singleton INSTANCE = new Singleton(); // safe!
}
```

```
// PITFALL 3 – No happens-before across threads without sync
int x = 0;
Thread t1 = new Thread(() -> x = 42);
Thread t2 = new Thread(() -> System.out.println(x)); // may print 0!
t1.start(); t2.start();
// No synchronization = no happens-before = t2 may see stale x
```

PENDING 13: Performance — Context Switching Cost

Never discussed:

Context Switch cost: ~1-10 microseconds per switch

Platform thread:

- Stack size: ~1MB
- Creation cost: ~100 microseconds
- Context switch: ~1-10 microseconds
- Max practical: ~10,000 threads

Virtual thread (Java 21):

- Stack size: ~few KB (grows dynamically)
- Creation cost: ~microseconds
- Context switch: handled by JVM scheduler
- Max practical: millions

Implications:

```
// Thread-per-request model – COSTLY at scale
// 10,000 concurrent requests = 10,000 threads = 10GB RAM

// Thread pool model – better
// 200 threads handle 10,000 requests via queuing

// Virtual thread model (Java 21) – best for I/O
// 10,000 virtual threads = minimal memory, no OS context switches
```

🔴 PENDING 14: `CompletableFuture` — Exception Propagation Rules

```
CompletableFuture<String> cf =
    CompletableFuture.supplyAsync(() -> {
        throw new RuntimeException("BOOM");
    });

// Exception wrapped in CompletionException
cf.join(); // throws CompletionException wrapping RuntimeException

// Get original cause
try {
    cf.join();
} catch (CompletionException e) {
    Throwable cause = e.getCause(); // original RuntimeException
}

// Exception propagation through chain
CompletableFuture
    .supplyAsync(() -> { throw new RuntimeException("fail"); })
    .thenApply(r -> r + "!")           // SKIPPED – exception propagates
    .thenApply(r -> r.toUpperCase()) // SKIPPED
    .exceptionally(ex -> "recovered") // CATCHES – returns "recovered"
    .thenApply(r -> r + "?")         // RUNS – gets "recovered?"
```

🔴 PENDING 15: Parallel Stream — `Spliterator`

```
// Spliterator – how parallel streams split work
// Custom Spliterator for custom data structures

class RangeSpliterator implements Spliterator<Integer> {
    private int start, end;

    public boolean tryAdvance(Consumer<? super Integer> action) {
        if (start < end) { action.accept(start++); return true; }
        return false;
    }
}
```

```

public Spliterator<Integer> trySplit() {
    int mid = (start + end) / 2;
    if (mid == start) return null; // too small to split
    RangeSpliterator prefix = new RangeSpliterator(start, mid);
    this.start = mid;
    return prefix; // ForkJoinPool uses this to divide work
}

public long estimateSize() { return end - start; }
public int characteristics() { return ORDERED | SIZED | SUBSIZED; }
}

```

FINAL COMPLETE CHECKLIST

Fully Covered

synchronized, volatile, Atomic classes, LongAdder
 ReentrantLock, ReadWriteLock, StampedLock (comparison)
 CountdownLatch, CyclicBarrier, Semaphore, Phaser (basic)
 CompletableFuture (full API), Future
 ThreadPoolExecutor, ForkJoinPool, ScheduledExecutor
 ConcurrentHashMap, CopyOnWriteArrayList (comparison)
 BlockingQueue variants, DelayQueue, SynchronousQueue
 ThreadLocal, InheritableThreadLocal
 JMM, Happens-Before rules
 Thread lifecycle, Daemon, Priority
 wait/notify, sleep vs wait
 Thread interruption
 Virtual Threads, Structured Concurrency
 Anti-patterns, Debugging
 Java 8 additions complete

Still Pending

1. Biased/Thin/Fat Lock JVM internals
2. ReentrantReadWriteLock full API + lock downgrade
3. Phaser deep dive
4. BlockingDeque
5. Lock Conditions (newCondition)

6. ThreadFactory
7. UncaughtExceptionHandler
8. TransferQueue
9. final field memory semantics
10. CopyOnWriteArrayList internals
11. Happens-Before pitfalls
12. Context switching cost
13. CompletableFuture exception propagation rules
14. Spliterator / parallel stream internals

Want me to cover all 14 pending topics now?

4/23/2026, 8:38:05 AM

 yes any scenario based questions missed add it here also

4/23/2026, 8:39:25 AM

Here are all **14 pending topics + missed scenario-based questions**:

PENDING 1: Biased / Thin / Fat Lock — JVM Internals

How JVM Manages Locks Internally

Every Java object has an **object header** (mark word — 64 bits) that stores lock state:

Object Header (Mark Word):

Biased Lock		hash		age		biased_lock		lock bits	
-------------	--	------	--	-----	--	-------------	--	-----------	--

3 Lock States

State 1 — Biased Lock (No contention ever)

Assumption: **only ONE** thread ever accesses this object
JVM stamps the thread ID **into** object header
Next **time** same thread locks → **no** CAS, **no atomic** op needed
Just **check**: "is my thread ID in the header?" → YES → proceed

Cost: Near ZERO — just a memory read comparison

Revoked **when**: a **second** thread tries **to** acquire → bias revoked
JVM stops **all** threads (STW) **to revoke** bias

State 2 — Thin Lock (Low contention)

Two threads accessing but **not** at the same time
Uses CAS **on** mark word — no OS involvement
Thread spins briefly waiting **for lock**

Cost: CAS operation – very fast

Escalates **when**: thread spins too **long** without getting **lock**

State 3 — Fat Lock (High contention)

Multiple threads actively contending

JVM inflates **lock** → creates OS **mutex** (monitor)

Losing threads SLEEP – OS scheduler wakes them

Context **switch** involved – expensive

Cost: OS-level – microseconds per operation

Escalation Path

NEW OBJECT

↓

BIASED LOCK ← only **1** thread ever touches it

↓ (**2nd** thread appears)

THIN LOCK ← low contention, CAS spin

↓ (high contention, **long** spin)

FAT LOCK ← OS mutex, threads sleep

Practical Impact

```
// Biased locking – good for single-threaded access patterns
// e.g., StringBuilder used locally (never shared)
StringBuilder sb = new StringBuilder(); // biased to current thread
sb.append("a"); // lock/unlock near zero cost
sb.append("b");

// Disable biased locking (sometimes better for highly concurrent apps)
// -XX:-UseBiasedLocking
// Avoids STW bias revocation pauses

// Java 15+ – biased locking deprecated
// Java 21 – biased locking removed entirely
```

PENDING 2: **ReentrantReadWriteLock** — Complete API

```
ReentrantReadWriteLock rwLock = new ReentrantReadWriteLock(true); // fair=true
Lock readLock = rwLock.readLock();
Lock writeLock = rwLock.writeLock();
```

Read Lock — Multiple Readers Simultaneously

```
readLock.lock();
try {
    return sharedData; // many threads can read at same time
} finally {
    readLock.unlock();
}
```

Write Lock — Exclusive Access

```
writeLock.lock();
try {
    sharedData = newValue; // only ONE thread writes, blocks all readers
} finally {
    writeLock.unlock();
}
```

Lock Downgrade — Write → Read (Allowed)

```
writeLock.lock();
try {
    sharedData = compute(); // update data
    readLock.lock();       // acquire read BEFORE releasing write
} finally {
    writeLock.unlock();    // release write – now holding only read
}
try {
    return sharedData;     // read with read lock
} finally {
    readLock.unlock();
}
// Why downgrade? – ensures you read YOUR OWN write
// without another writer sneaking in between
```

Lock Upgrade — Read → Write (NOT Allowed)

```
// DEADLOCK – never do this
readLock.lock();
try {
    writeLock.lock(); // DEADLOCK – two threads both try this = neither proceeds
} finally {
    readLock.unlock();
}
// Fix: release read lock first, then acquire write lock
// But then another writer may modify data between the two operations
// Use StampedLock.tryConvertToWriteLock() instead
```

Monitoring Methods

```
rwLock.getReadLockCount()           // how many threads hold read lock
rwLock.getReadHoldCount()           // how many times THIS thread holds read lock
rwLock.isWriteLocked()               // is write lock held?
rwLock.isWriteLockedByCurrentThread() // does THIS thread hold write lock?
rwLock.getQueueLength()              // threads waiting for either lock
rwLock.hasQueuedThreads()            // any threads waiting?
rwLock.hasQueuedThread(t)            // specific thread waiting?
```

Real Use Case — Price Cache

```
class PriceCache {
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();
    private final Map<String, Double> prices = new HashMap<>();

    public Double getPrice(String symbol) {
        lock.readLock().lock(); // many threads can read simultaneously
        try {
            return prices.get(symbol);
        } finally {
            lock.readLock().unlock();
        }
    }

    public void updatePrice(String symbol, double price) {
        lock.writeLock().lock(); // exclusive – all readers blocked
        try {
            prices.put(symbol, price);
        } finally {
            lock.writeLock().unlock();
        }
    }
}
```

```
    }  
  }  
}
```

PENDING 3: **Phaser** — Deep Dive

Basic Usage

```
Phaser phaser = new Phaser(3); // 3 registered parties  
  
Runnable task = () -> {  
    System.out.println("Phase 0 work");  
    phaser.arriveAndAwaitAdvance(); // barrier – wait for all 3  
  
    System.out.println("Phase 1 work");  
    phaser.arriveAndAwaitAdvance(); // barrier again  
  
    System.out.println("Phase 2 work");  
    phaser.arriveAndDeregister(); // done – reduce party count  
};  
  
new Thread(task).start();  
new Thread(task).start();  
new Thread(task).start();
```

Key Methods

```
phaser.arrive()           // arrive but DON'T wait for others  
phaser.arriveAndAwaitAdvance() // arrive AND wait for all others  
phaser.arriveAndDeregister() // arrive, deregister, don't wait  
phaser.awaitAdvance(phase) // wait for phase to complete (without arriving)  
phaser.register()         // add 1 new party dynamically  
phaser.bulkRegister(5)    // add 5 parties dynamically  
phaser.getPhase()         // current phase number (0,1,2...)  
phaser.getRegisteredParties() // total registered  
phaser.getArrivedParties() // arrived so far this phase  
phaser.getUnarrivedParties() // yet to arrive this phase
```

```
phaser.forceTermination()    // terminate phaser
phaser.isTerminated()        // check if terminated
```

Override `onAdvance()` — Custom Phase Logic

```
Phaser phaser = new Phaser(3) {
    @Override
    protected boolean onAdvance(int phase, int registeredParties) {
        System.out.println("Phase " + phase + " complete!");

        // Return true = terminate phaser after this advance
        // Return false = continue to next phase
        return phase >= 2 || registeredParties == 0; // stop after phase 2
    }
};
```

Dynamic Party Registration

```
Phaser phaser = new Phaser(1); // 1 party = main thread

for (int i = 0; i < 5; i++) {
    phaser.register(); // register BEFORE starting thread
    new Thread(() -> {
        doWork();
        phaser.arriveAndDeregister(); // deregister when done
    }).start();
}

phaser.arriveAndAwaitAdvance(); // main thread waits for all
System.out.println("All done!");
```

Phaser vs CyclicBarrier vs CountdownLatch

CountDownLatch → one phase, **fixed** count, **not** reusable
CyclicBarrier → multiple phases, **fixed** parties, reusable
Phaser → multiple phases, **dynamic** parties, reusable, tiered

PENDING 4: `BlockingDeque`

```

// Double-ended blocking queue
BlockingDeque<Task> deque = new LinkedBlockingDeque<>(100);

// Add to FRONT (head)
deque.putFirst(task);           // blocks if full
deque.offerFirst(task);         // returns false if full
deque.offerFirst(task, 1, SECONDS); // waits up to 1 second

// Add to END (tail)
deque.putLast(task);           // blocks if full
deque.offerLast(task);         // returns false if full

// Remove from FRONT
Task t = deque.takeFirst();     // blocks if empty
Task t = deque.pollFirst();     // returns null if empty
Task t = deque.pollFirst(1, SECONDS); // waits up to 1 second

// Remove from END
Task t = deque.takeLast();      // blocks if empty
Task t = deque.pollLast();      // returns null if empty

// Peek without removing
deque.peekFirst();
deque.peekLast();

```

Work Stealing Pattern

```

// Each worker has own deque
Map<Integer, BlockingDeque<Task>> workerQueues = new ConcurrentHashMap<>();

class Worker implements Runnable {
    int id;
    BlockingDeque<Task> myDeque = workerQueues.get(id);

    public void run() {
        while (true) {
            Task task = myDeque.pollFirst(); // take from own front

            if (task == null) {
                // Steal from another worker's BACK
                task = stealFromOthers();
            }
        }
    }
}

```

```

        }

        if (task != null) process(task);
    }
}

Task stealFromOthers() {
    for (BlockingDeque<Task> otherDeque : workerQueues.values()) {
        Task stolen = otherDeque.pollLast(); // steal from tail
        if (stolen != null) return stolen;
    }
    return null;
}
}

```

PENDING 5: Lock **Condition** — Deep Dive

Basic Usage

```

ReentrantLock lock = new ReentrantLock();
Condition notFull = lock.newCondition();
Condition notEmpty = lock.newCondition();

```

Bounded Buffer Using Conditions

```

class BoundedBuffer<T> {
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    public void put(T item) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == capacity)
                notFull.await(); // wait - releases lock
            queue.add(item);
        }
    }
}

```

```

        notEmpty.signal();           // wake ONE consumer
    } finally { lock.unlock(); }
}

public T take() throws InterruptedException {
    lock.lock();
    try {
        while (queue.isEmpty())
            notEmpty.await();         // wait – releases lock
        T item = queue.poll();
        notFull.signal();             // wake ONE producer
        return item;
    } finally { lock.unlock(); }
}
}

```

All Condition Methods

```

condition.await()           // wait, releases lock, reacquires on wake
condition.await(1, SECONDS) // timed wait
condition.awaitNanos(1000)  // nanosecond wait
condition.awaitUninterruptibly() // wait, ignore interruptions
condition.awaitUntil(deadline) // wait until absolute Date
condition.signal()           // wake ONE waiting thread
condition.signalAll()        // wake ALL waiting threads

```

signal() vs signalAll()

signal() → wakes **ONE** random waiter → efficient but risky
 (wrong thread may wake up – use **only when all** waiters
 check same **condition**)

signalAll() → wakes **ALL** waiters → safe but more context switches
 (each woken thread re-checks **condition in** while loop)

Condition vs Object.wait/notify

Object.wait() → **ONE** wait **set per** object
Condition → **MULTIPLE** wait sets **per** lock
 e.g., notFull **and** notEmpty separate

```
consumers don't accidentally wake producers
producers don't accidentally wake producers
```

PENDING 6: ThreadFactory

```
// Default threads have names like: pool-1-thread-1
// Hard to debug in production thread dumps
// ThreadFactory gives full control

public class NamedThreadFactory implements ThreadFactory {
    private final String namePrefix;
    private final boolean daemon;
    private final int priority;
    private final AtomicInteger count = new AtomicInteger(0);

    public NamedThreadFactory(String namePrefix, boolean daemon, int priority) {
        this.namePrefix = namePrefix;
        this.daemon = daemon;
        this.priority = priority;
    }

    @Override
    public Thread newThread(Runnable r) {
        Thread t = new Thread(r);
        t.setName(namePrefix + "-" + count.incrementAndGet());
        t.setDaemon(daemon);
        t.setPriority(priority);
        t.setUncaughtExceptionHandler((thread, ex) ->
            LoggerFactory.getLogger(thread.getName())
                .error("Uncaught exception", ex)
        );
        return t;
    }
}

// Usage
ExecutorService orderPool = Executors.newFixedThreadPool(
    10,
    new NamedThreadFactory("order-processor", false, Thread.NORM_PRIORITY)
```

```
);

ExecutorService monitorPool = Executors.newFixedThreadPool(
    2,
    new NamedThreadFactory("system-monitor", true, Thread.MIN_PRIORITY)
);
```

Thread Dump — With vs Without Custom Factory

WITHOUT factory:

```
pool-1-thread-1  BLOCKED
pool-1-thread-2  WAITING
pool-2-thread-1  RUNNABLE
```

Debugging: impossible

WITH factory:

```
order-processor-1  BLOCKED
order-processor-2  WAITING
system-monitor-1  RUNNABLE
```

Debugging: clear which pool has issue

PENDING 7: `UncaughtExceptionHandler`

Problem — Silent Exception Swallowing

```
ExecutorService pool = Executors.newFixedThreadPool(5);

pool.submit(() -> {
    throw new RuntimeException("Critical failure!"); // SILENTLY LOST!
});

// No exception in logs, no alert – task just disappears
```

Solution 1 — Per-Thread Handler

```
Thread t = new Thread(() -> riskyWork());
t.setUncaughtExceptionHandler((thread, ex) -> {
    log.error("Thread {} failed: {}", thread.getName(), ex.getMessage(), ex);
    alertOpsTeam(ex);
    metrics.incrementErrorCount();
});
t.start();
```

Solution 2 — Global Default Handler

```
// Catches ALL unhandled exceptions across all threads
Thread.setDefaultUncaughtExceptionHandler((thread, ex) -> {
    log.error("UNHANDLED EXCEPTION in thread {}", thread.getName(), ex);
    // send to monitoring system
    Sentry.captureException(ex);
});
```

Solution 3 — Custom ThreadPoolExecutor with afterExecute

```
class SafeThreadPool extends ThreadPoolExecutor {

    public SafeThreadPool(int size) {
        super(size, size, 0, MILLISECONDS, new LinkedBlockingQueue<>());
    }

    @Override
    protected void afterExecute(Runnable r, Throwable t) {
        super.afterExecute(r, t);

        // t is null for submit() - exception hidden in Future
        if (t == null && r instanceof Future<?>) {
            try {
                ((Future<?>) r).get();
            } catch (ExecutionException ee) {
                t = ee.getCause();
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();
            }
        }

        if (t != null) {
            log.error("Task failed with exception: ", t);
            metrics.recordFailure(t);
        }
    }

    @Override
    protected void beforeExecute(Thread t, Runnable r) {
        log.debug("Task starting on thread: {}", t.getName());
    }
}
```

Solution 4 — Wrap submit() with try-catch

```
pool.submit(() -> {
    try {
        riskyWork();
    } catch (Exception e) {
        log.error("Task failed", e);
        // handle or rethrow as needed
    }
});
```

PENDING 8: **TransferQueue**

```
TransferQueue<String> queue = new LinkedTransferQueue<>();

// PRODUCER — two modes:

// Mode 1: put() — add to queue, don't wait for consumer
queue.put("order-123"); // returns immediately

// Mode 2: transfer() — BLOCK until consumer takes it directly
queue.transfer("order-456"); // blocks until a consumer calls take()
// Guarantees consumer HAS the item before producer continues

// Mode 3: tryTransfer() — non-blocking
boolean handed = queue.tryTransfer("order-789"); // false if no waiting consumer

// Mode 4: tryTransfer with timeout
boolean handed = queue.tryTransfer("order-101", 500, MILLISECONDS);

// CONSUMER
String order = queue.take(); // blocks if empty
String order = queue.poll(); // null if empty
String order = queue.poll(1, SECONDS); // wait up to 1s

// Check consumer status
int waiting = queue.getWaitingConsumerCount();
boolean has = queue.hasWaitingConsumer();
```

Use Case — Acknowledgment Pattern

```
// Producer must know consumer received the message
TransferQueue<Task> queue = new LinkedTransferQueue<>();

// Producer
queue.transfer(criticalTask); // BLOCKS until consumer takes it
log.info("Consumer confirmed receipt – safe to proceed");

// Consumer
Task task = queue.take();
processTask(task); // producer is now unblocked
```

TransferQueue vs Others

```
put()      → add to queue, producer continues immediately
transfer() → producer WAITS for consumer direct handoff
SynchronousQueue → ALWAYS waits (like transfer() only)
LinkedTransferQueue → CHOICE: wait or not wait per message
```

PENDING 9: `final` Field Memory Semantics

JMM Guarantee for `final` Fields

```
class SafePublication {
    final int x;
    final int y;
    int z; // NOT final

    SafePublication(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}

// Published WITHOUT synchronization
SafePublication obj = new SafePublication(1, 2, 3);
```

```
sharedRef = obj; // publish to other threads

// Thread 2 reads:
SafePublication o = sharedRef;
System.out.println(o.x); // GUARANTEED: 1 – final field
System.out.println(o.y); // GUARANTEED: 2 – final field
System.out.println(o.z); // NOT GUARANTEED – may see 0! not final
```

Why final Fields Are Special

JMM gives **final** fields a special "freeze" action:

- After constructor completes, **all final** field writes **are** FROZEN
- **Any** thread reading the object sees correct **final values**
- Even **without** synchronization **or** volatile

Non-**final** fields have **NO** such guarantee

Without sync – another thread may see partially constructed object

Immutable Objects — Thread Safe Without Sync

```
// Fully immutable – thread safe to share without any synchronization
public final class Money {
    private final long amount;      // final
    private final String currency; // final

    public Money(long amount, String currency) {
        this.amount = amount;
        this.currency = currency;
    }
    // only getters – no setters
}

// Safe to share across threads – no volatile, no synchronized needed
Money price = new Money(100, "USD");
// All threads see amount=100, currency="USD" – guaranteed
```

Escape During Construction — BREAKS Final Guarantee

```
class Broken {
    final int value;
    static Broken instance; // shared reference
```

```
Broken() {
    instance = this; // ESCAPE – published before constructor finishes!
    this.value = 42; // too late – other thread may see value=0
}
}
// NEVER publish 'this' reference from constructor
// Includes: registering as listener, starting threads in constructor
```

PENDING 10: `CopyOnWriteArrayList` — Internal Working

How It Works Internally

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
```

Write Operations — Copy on Every Write

```
list.add("A");
// Internally:
// 1. Acquire ReentrantLock
// 2. Copy entire array: ["A"] – new array
// 3. Add element: ["A", "B"]
// 4. Atomically replace array reference (volatile write)
// 5. Release lock

list.remove("A");
// Same: copy → modify copy → replace reference → release

// Writes are MUTUALLY EXCLUSIVE – only one writer at a time
// Writes are EXPENSIVE – O(n) copy for every write
```

Read Operations — Always Lock-Free

```
list.get(0);
list.size();
list.contains("A");
list.iterator();
```

```
// All reads just access the current array reference – NO LOCK
// volatile array reference ensures visibility
```

Iterator — Snapshot Semantics

```
list.add("A");
list.add("B");
list.add("C");

Iterator<String> it = list.iterator(); // takes SNAPSHOT of current array

list.add("D"); // modify list AFTER iterator created
list.remove("A");

// Iterator still sees: A, B, C – original snapshot
// No ConcurrentModificationException – ever
while (it.hasNext()) {
    System.out.println(it.next()); // A, B, C
}
// "D" never seen by this iterator
```

Internal Array Reference

```
// Simplified internal implementation
public class CopyOnWriteArrayList<E> {
    private volatile Object[] array; // volatile – ensures visibility
    final transient ReentrantLock lock = new ReentrantLock();

    public boolean add(E e) {
        final ReentrantLock lock = this.lock;
        lock.lock();
        try {
            Object[] elements = array;
            int len = elements.length;
            Object[] newElements = Arrays.copyOf(elements, len + 1); // COPY
            newElements[len] = e;
            array = newElements; // atomic reference replacement
            return true;
        } finally {
            lock.unlock();
        }
    }
}
```

```

public E get(int index) {
    return (E) array[index]; // direct array access – no lock!
}
}

```

When to Use / Avoid

- ✅ Use **when**:
 - Reads >> Writes
 - List **is** small
 - Need iteration **without** ConcurrentModificationException
 - Event listeners, observers, hooks
- ❌ Avoid **when**:
 - Frequent writes ($O(n)$ **copy every time**)
 - **Large** list (copying **1M** elements **on every add**)
 - Need **real-time** consistency during iteration
 - Memory **is** constrained (two copies exist during write)

🔴 PENDING 11: Happens-Before Pitfalls

Pitfall 1 — Reordering Without Volatile

```

int value = 0;
boolean ready = false; // NOT volatile

// Thread 1
value = 42; // write 1
ready = true; // write 2
// JIT/CPU can REORDER: ready=true may appear BEFORE value=42

// Thread 2
if (ready) {
    System.out.println(value); // may print 0! reordering!
}

// FIX
volatile boolean ready = false;

```

```
// volatile write to ready creates happens-before  
// ALL writes before volatile write are visible after volatile read
```

Pitfall 2 — Synchronized on Different Objects

```
Object lock1 = new Object();  
Object lock2 = new Object();  
int data = 0;  
  
// Thread 1  
synchronized (lock1) { data = 42; } // happens-before ONLY for lock1  
  
// Thread 2  
synchronized (lock2) { System.out.println(data); } // DIFFERENT lock!  
// No happens-before relationship – may see data=0  
  
// FIX – synchronize on SAME object  
synchronized (lock1) { data = 42; }  
synchronized (lock1) { System.out.println(data); }
```

Pitfall 3 — Thread.sleep() Does NOT Create Happens-Before

```
int data = 0;  
  
Thread t1 = new Thread(() -> data = 42);  
t1.start();  
Thread.sleep(1000); // WRONG – does not guarantee visibility of data=42  
  
System.out.println(data); // may still see 0!  
  
// FIX  
t1.join(); // join() creates happens-before  
System.out.println(data); // guaranteed to see 42
```

Pitfall 4 — Piggybacking Incorrectly

```
// WRONG – trying to piggyback on unrelated volatile  
volatile int unrelated = 0;  
int importantData = 0;  
  
// Thread 1  
importantData = 42;
```

```

unrelated = 1; // volatile write

// Thread 2
if (unrelated == 1) {
    // importantData IS visible – piggyback works
    // BUT this is fragile and hard to maintain
    // clearly document the dependency
}

// BETTER – use dedicated volatile flag
volatile boolean dataReady = false;
importantData = 42;
dataReady = true; // explicit, clear intent

```

Pitfall 5 — Long/Double Non-Atomic Reads

```

// On 32-bit JVM – long/double reads/writes NOT atomic!
long sharedLong = 0;

// Thread 1 writes: 0xFFFFFFFF_FFFFFFFF
// Thread 2 may read: 0x00000000_FFFFFFFF (torn read – only half written)

// FIX
volatile long sharedLong = 0; // volatile guarantees atomic 64-bit read/write
// Or use AtomicLong

```

PENDING 12: Context Switching Cost

What is a Context Switch

CPU can only run ONE thread at a time per core.
When OS switches from Thread A to Thread B:

1. Save Thread A state:
 - Program counter
 - CPU registers
 - Stack pointer

2. Load Thread B state:

- Restore its registers
- Restore stack pointer
- Jump to its program counter

Cost: ~1 to 10 microseconds per switch

+ cache pollution (Thread B evicts Thread A's data from L1/L2 cache)

Voluntary vs Involuntary Context Switch

Voluntary: Thread calls `sleep()`, `wait()`, I/O, `lock.lock()` (blocked)
Thread yields CPU willingly

Involuntary: OS time-slice `expires` (typically 10-100ms)
Higher priority thread preempts current thread
More expensive – happens mid-computation

Measuring Context Switches

Linux – context switches per second

`vmstat 1`

Per-process context switches

`cat /proc/<pid>/status | grep ctxt`

Java – voluntary vs involuntary

`jcmd <pid> Thread.print`

Minimizing Context Switches

// 1. Right-size thread pools

// More threads than cores on CPU-bound work = more context switches

`int cores = Runtime.getRuntime().availableProcessors();`

`ExecutorService pool = Executors.newFixedThreadPool(cores); // CPU-bound`

// 2. Avoid frequent blocking

// Bad – blocks thread → context switch

`synchronized (lock) { Thread.sleep(100); }`

// 3. Use non-blocking algorithms

`AtomicInteger counter = new AtomicInteger();`

`counter.incrementAndGet(); // no context switch – CAS in CPU`

```
// 4. Batch work to reduce scheduling overhead
// Bad – 1000 tiny tasks = 1000 schedule/context switches
for (int i = 0; i < 1000; i++) pool.submit(() -> doTinyWork());

// Better – 10 batched tasks
for (int i = 0; i < 10; i++) {
    final int batch = i;
    pool.submit(() -> {
        for (int j = batch*100; j < (batch+1)*100; j++) doTinyWork();
    });
}

// 5. Virtual threads (Java 21) – no OS context switch
Thread.ofVirtual().start(() -> doWork()); // JVM-managed, cheap
```

PENDING 13: `CompletableFuture` Exception Propagation

Exception Wrapping

```
CompletableFuture<String> cf =
    CompletableFuture.supplyAsync(() -> {
        throw new IllegalArgumentException("bad input");
    });

// join() wraps in CompletionException
try {
    cf.join();
} catch (CompletionException e) {
    Throwable cause = e.getCause(); // IllegalArgumentException
    System.out.println(cause.getMessage()); // "bad input"
}

// get() wraps in ExecutionException
try {
    cf.get();
} catch (ExecutionException e) {
    Throwable cause = e.getCause(); // IllegalArgumentException
}
```

Exception Skips Transformation Stages

```
CompletableFuture
    .supplyAsync(() -> {
        throw new RuntimeException("BOOM");
    })
    .thenApply(r -> r + "!")           // SKIPPED – exception propagates
    .thenApply(r -> r.toUpperCase()) // SKIPPED
    .thenAccept(System.out::println) // SKIPPED
    .exceptionally(ex -> {
        System.out.println("Caught: " + ex.getMessage()); // RUNS
        return null;
    });
```

exceptionally vs handle vs whenComplete

```
CompletableFuture<String> failing =
    CompletableFuture.supplyAsync(() -> { throw new RuntimeException("fail"); });

// exceptionally – ONLY on failure, provides fallback
failing
    .exceptionally(ex -> "fallback") // returns "fallback"
    .thenAccept(System.out::println); // prints "fallback"

// handle – BOTH success and failure, can transform
failing
    .handle((result, ex) -> {
        if (ex != null) return "recovered: " + ex.getMessage();
        return result;
    })
    .thenAccept(System.out::println); // prints "recovered: fail"

// whenComplete – BOTH, side-effect only, result unchanged
failing
    .whenComplete((result, ex) -> {
        if (ex != null) log.error("Failed", ex); // logging only
    })
    .exceptionally(ex -> "still need to handle"); // exception still propagates!
```

Exception in Callback Itself

```
CompletableFuture
    .supplyAsync(() -> "hello")
    .thenApply(r -> {
        throw new RuntimeException("callback failed!"); // exception in thenApply
    })
    .exceptionally(ex -> {
        // catches BOTH upstream exception AND callback exception
        return "caught: " + ex.getMessage();
    });
```

Re-throwing in Handle

```
// handle() can re-throw to propagate as new exception
.handle((result, ex) -> {
    if (ex != null) {
        if (ex.getCause() instanceof TimeoutException) {
            throw new CompletionException(new RetryableException(ex));
        }
        throw new CompletionException(ex); // re-wrap and propagate
    }
    return result;
});
```

PENDING 14: **Spliterator** / Parallel Stream Internals

What is a Spliterator

```
// Spliterator = Splittable Iterator
// Parallel streams use Spliterator to DIVIDE work across threads

// Every Collection has a default spliterator
Spliterator<String> sp = list.spliterator();

// tryAdvance – process one element
sp.tryAdvance(item -> System.out.println(item));

// forEachRemaining – process all remaining
sp.forEachRemaining(System.out::println);
```

```
// trySplit – DIVIDE into two halves (key for parallelism)
Spliterator<String> firstHalf = sp.trySplit(); // sp now has second half

sp.estimateSize();      // estimated elements remaining
sp.characteristics();    // flags: ORDERED, SIZED, DISTINCT, SORTED etc.
```

How Parallel Stream Uses Spliterator

```
list.parallelStream().map(fn).collect(toList());

// Internally:
// 1. Get Spliterator from list
// 2. ForkJoinPool tries to split repeatedly
//    list(1000) → [0-499] + [500-999]
//              → [0-249] + [250-499] + [500-749] + [750-999]
//              → ... until THRESHOLD reached
// 3. Each piece submitted as ForkJoinTask
// 4. Results merged back up the tree
```

Custom Spliterator

```
class RangeSpliterator implements Spliterator<Integer> {
    private int start, end;
    private static final int THRESHOLD = 100;

    RangeSpliterator(int start, int end) {
        this.start = start;
        this.end = end;
    }

    @Override
    public boolean tryAdvance(Consumer<? super Integer> action) {
        if (start < end) {
            action.accept(start++);
            return true;
        }
        return false;
    }

    @Override
    public Spliterator<Integer> trySplit() {
        int size = end - start;
```

```

    if (size <= THRESHOLD) return null; // too small – don't split

    int mid = start + size / 2;
    RangeSplitter left = new RangeSplitter(start, mid);
    this.start = mid; // this becomes right half
    return left;
}

@Override
public long estimateSize() { return end - start; }

@Override
public int characteristics() {
    return ORDERED | SIZED | SUBSIZED | IMMUTABLE;
}
}

// Use with stream
StreamSupport.stream(new RangeSplitter(0, 1000), true) // true=parallel
    .map(n -> n * n)
    .sum();

```

Spliterator Characteristics — Affect Optimizations

```

ORDERED    → elements have defined order (List – yes, HashSet – no)
DISTINCT   → no duplicates (Set – yes)
SORTED     → elements are sorted (TreeSet – yes)
SIZED      → knows exact size upfront (ArrayList – yes, filter result – no)
SUBSIZED   → splits also know their size (ArrayList – yes, LinkedList – no)
NONNULL    → no null elements
IMMUTABLE  → source won't change during traversal
CONCURRENT → source supports concurrent modification

```

MISSED SCENARIO-BASED QUESTIONS

SCENARIO 1: Two Threads Print Odd and Even Numbers Alternately

```

// Classic interview scenario
// Thread 1 prints: 1, 3, 5, 7...
// Thread 2 prints: 2, 4, 6, 8...

class OddEvenPrinter {
    private final Object lock = new Object();
    private volatile int number = 1;
    private final int limit;

    OddEvenPrinter(int limit) { this.limit = limit; }

    void printOdd() throws InterruptedException {
        while (number <= limit) {
            synchronized (lock) {
                while (number % 2 == 0)    // not my turn
                    lock.wait();
                if (number <= limit)
                    System.out.println("ODD: " + number);
                number++;
                lock.notifyAll();
            }
        }
    }

    void printEven() throws InterruptedException {
        while (number <= limit) {
            synchronized (lock) {
                while (number % 2 != 0)    // not my turn
                    lock.wait();
                if (number <= limit)
                    System.out.println("EVEN: " + number);
                number++;
                lock.notifyAll();
            }
        }
    }
}

OddEvenPrinter printer = new OddEvenPrinter(10);
new Thread(() -> printer.printOdd()).start();
new Thread(() -> printer.printEven()).start();

```

SCENARIO 2: Print Numbers 1-N Using 3 Threads in Order

```
// Thread 1 prints: 1, 4, 7...
// Thread 2 prints: 2, 5, 8...
// Thread 3 prints: 3, 6, 9...

class ThreeThreadPrinter {
    private final Object lock = new Object();
    private volatile int current = 1;
    private final int limit;
    private final int totalThreads = 3;

    ThreeThreadPrinter(int limit) { this.limit = limit; }

    void print(int threadNum) throws InterruptedException {
        while (true) {
            synchronized (lock) {
                while (current <= limit && (current % totalThreads) != (threadNum %
totalThreads))
                    lock.wait();
                if (current > limit) break;
                System.out.println("Thread-" + threadNum + ": " + current);
                current++;
                lock.notifyAll();
            }
        }
    }
}
```

SCENARIO 3: Implement **BlockingQueue** from Scratch

```
class MyBlockingQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notFull = lock.newCondition();
    private final Condition notEmpty = lock.newCondition();

    MyBlockingQueue(int capacity) { this.capacity = capacity; }

    public void put(T item) throws InterruptedException {
```

```

        lock.lock();
        try {
            while (queue.size() == capacity)
                notFull.await();           // wait for space
            queue.add(item);
            notEmpty.signal();             // signal consumer
        } finally { lock.unlock(); }
    }

    public T take() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty())
                notEmpty.await();           // wait for item
            T item = queue.poll();
            notFull.signal();               // signal producer
            return item;
        } finally { lock.unlock(); }
    }

    public int size() {
        lock.lock();
        try { return queue.size(); }
        finally { lock.unlock(); }
    }
}

```

SCENARIO 4: N Threads Download Files, One Thread Merges After All Done

```

int fileCount = 5;
CountDownLatch latch = new CountDownLatch(fileCount);
List<String> downloadedFiles = new CopyOnWriteArrayList<>();

ExecutorService pool = Executors.newFixedThreadPool(fileCount);

for (int i = 0; i < fileCount; i++) {
    final int fileId = i;
    pool.submit(() -> {
        try {
            String file = downloadFile(fileId); // download
            downloadedFiles.add(file);
        }
    });
}

```

```

        } finally {
            latch.countDown(); // signal done – even if download failed
        }
    });
}

// Merger thread
new Thread(() -> {
    try {
        latch.await(); // wait for ALL downloads
        mergeFiles(downloadedFiles); // merge
        System.out.println("Merge complete!");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}).start();

```

SCENARIO 5: Simulate Deadlock and Fix It

```

// DEADLOCK
Object lockA = new Object();
Object lockB = new Object();

Thread t1 = new Thread(() -> {
    synchronized (lockA) { // holds A, wants B
        Thread.sleep(100);
        synchronized (lockB) { doWork(); }
    }
});

Thread t2 = new Thread(() -> {
    synchronized (lockB) { // holds B, wants A
        Thread.sleep(100);
        synchronized (lockA) { doWork(); } // DEADLOCK!
    }
});

// FIX 1 – consistent lock ordering
Thread t1Fixed = new Thread(() -> {
    synchronized (lockA) {
        synchronized (lockB) { doWork(); } // always A then B
    }
});

```

```

    }
});
Thread t2Fixed = new Thread(() -> {
    synchronized (lockA) { // same order – A then B
        synchronized (lockB) { doWork(); }
    }
});

// FIX 2 – tryLock with timeout
ReentrantLock la = new ReentrantLock();
ReentrantLock lb = new ReentrantLock();

boolean acquiredA = la.tryLock(1, SECONDS);
boolean acquiredB = lb.tryLock(1, SECONDS);
if (acquiredA && acquiredB) {
    try { doWork(); }
    finally { la.unlock(); lb.unlock(); }
} else {
    if (acquiredA) la.unlock();
    if (acquiredB) lb.unlock();
    // retry later
}

```

SCENARIO 6: Thread-Safe Lazy Initialization of Expensive Resource

```

// Pattern 1 – volatile double-checked locking
class ExpensiveResource {
    private static volatile ExpensiveResource instance;

    public static ExpensiveResource getInstance() {
        if (instance == null) {
            synchronized (ExpensiveResource.class) {
                if (instance == null) {
                    instance = new ExpensiveResource();
                }
            }
        }
        return instance;
    }
}

```

```
// Pattern 2 – Holder (BEST – no lock, no volatile)
class ExpensiveResource {
    private static class Holder {
        static final ExpensiveResource INSTANCE = new ExpensiveResource();
    }
    public static ExpensiveResource getInstance() {
        return Holder.INSTANCE; // class loaded lazily and safely
    }
}

// Pattern 3 – Enum singleton
enum ExpensiveResource {
    INSTANCE;
    // thread-safe, serialization-safe, reflection-safe
}
```

SCENARIO 7: Stop a Running Thread Gracefully

```
class Worker implements Runnable {
    private volatile boolean running = true; // volatile flag

    public void stop() {
        running = false;
    }

    @Override
    public void run() {
        while (running) {
            try {
                processNextItem();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); // restore flag
                break; // exit loop on interruption
            }
        }
        cleanup(); // graceful cleanup before exit
        System.out.println("Worker stopped gracefully");
    }
}

Worker worker = new Worker();
```

```
Thread t = new Thread(worker);
t.start();

// Stop gracefully – two ways:
worker.stop();    // via flag – for normal loops
t.interrupt();    // via interrupt – for blocking operations
```

SCENARIO 8: Multiple Producers Single Consumer

```
class MultiProducerSingleConsumer {
    private final BlockingQueue<Order> queue = new ArrayBlockingQueue<>(500);
    private final int producerCount;
    private final AtomicInteger finishedProducers = new AtomicInteger(0);

    void startProducer(int id) {
        new Thread(() -> {
            try {
                for (int i = 0; i < 100; i++) {
                    queue.put(new Order(id, i));
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            } finally {
                // Signal consumer when this producer is done
                if (finishedProducers.incrementAndGet() == producerCount) {
                    queue.put(Order.POISON_PILL); // signal end of stream
                }
            }
        }).start();
    }

    void startConsumer() {
        new Thread(() -> {
            while (true) {
                Order order = queue.take();
                if (order == Order.POISON_PILL) break; // all producers done
                processOrder(order);
            }
        }).start();
    }
}
```

SCENARIO 9: Timeout on Parallel Tasks — Cancel Others if One Succeeds

```
// First successful result wins – cancel all others
List<CompletableFuture<String>> futures = List.of(
    CompletableFuture.supplyAsync(() -> callServer1()),
    CompletableFuture.supplyAsync(() -> callServer2()),
    CompletableFuture.supplyAsync(() -> callServer3())
);

// anyOf – first to complete wins
CompletableFuture.anyOf(futures.toArray(new CompletableFuture[0]))
    .thenAccept(result -> {
        System.out.println("First result: " + result);
        // cancel remaining
        futures.forEach(f -> f.cancel(true));
    })
    .orTimeout(5, TimeUnit.SECONDS) // Java 9+
    .exceptionally(ex -> {
        System.out.println("All timed out or failed");
        return null;
    });
```

SCENARIO 10: Read-Heavy Config Map — Lock-Free Reads

```
// Config loaded once, refreshed rarely – thousands of reads per second
class ConfigStore {
    // AtomicReference – lock-free reads, safe writes
    private final AtomicReference<Map<String, String>> config =
        new AtomicReference<>(Collections.emptyMap());

    // READ – completely lock-free – thousands of threads, zero contention
    public String get(String key) {
        return config.get().get(key); // just two pointer dereferences
    }

    // WRITE – happens rarely (config refresh every 5 minutes)
    public void refresh(Map<String, String> newConfig) {
        config.set(Collections.unmodifiableMap(new HashMap<>(newConfig)));
        // atomic reference swap – readers instantly see new config
    }
}
```

```

    }

    // Conditional update – only if unchanged
    public boolean updateIfUnchanged(Map<String, String> expected,
                                     Map<String, String> newConfig) {
        return config.compareAndSet(expected,
                                     Collections.unmodifiableMap(new HashMap<>(newConfig)));
    }
}

```

🎯 Final Complete Coverage

✅ Now Fully Covered — All Topics

FUNDAMENTALS

- ✅ Thread lifecycle, creation, daemon, priority
- ✅ wait/notify/notifyAll, sleep vs wait
- ✅ Object-level vs class-level locking
- ✅ Thread interruption – complete handling
- ✅ volatile – deep dive + pitfalls
- ✅ JMM, happens-before – rules + pitfalls
- ✅ `final` field memory semantics
- ✅ Context switching cost + minimization

LOCKS

- ✅ synchronized – biased/thin/fat internals
- ✅ ReentrantLock – `full` API
- ✅ ReentrantReadWriteLock – `full` API + downgrade
- ✅ StampedLock – `full` API + optimistic read
- ✅ Lock Conditions (`newCondition`) – `full` API
- ✅ `LockSupport.park/unpark`

ATOMIC & NON-BLOCKING

- ✅ All `Atomic` classes
- ✅ LongAdder, LongAccumulator
- ✅ CAS internals
- ✅ ABA problem + `AtomicStampedReference`
- ✅ `False` sharing + `@Contended`

CONCURRENT COLLECTIONS

- ✅ `ConcurrentHashMap` – internals + Java 8 methods

- ✓ CopyOnWriteArrayList – internals
- ✓ All BlockingQueue variants
- ✓ BlockingDeque
- ✓ DelayQueue, TransferQueue, Exchanger
- ✓ ConcurrentSkipListMap/Set

COORDINATION

- ✓ CountdownLatch, CyclicBarrier – deep dive
- ✓ Phaser – deep dive
- ✓ Semaphore
- ✓ Exchanger

EXECUTOR FRAMEWORK

- ✓ ThreadPoolExecutor – all parameters
- ✓ ScheduledExecutorService
- ✓ ForkJoinPool + work stealing
- ✓ ThreadFactory
- ✓ UncaughtExceptionHandler
- ✓ Thread pool sizing formula
- ✓ Graceful shutdown

ASYNC

- ✓ Future – full API
- ✓ CompletableFuture – full API
- ✓ Exception propagation rules
- ✓ Async variants

JAVA 8 ADDITIONS

- ✓ CompletableFuture, LongAdder, StampedLock
- ✓ Parallel streams + Spliterator internals
- ✓ ConcurrentHashMap new methods
- ✓ Arrays.parallelSort/prefix/setAll

PATTERNS

- ✓ Producer-consumer
- ✓ Pipeline pattern
- ✓ Double-checked locking
- ✓ Holder pattern
- ✓ Read-heavy lock-free pattern
- ✓ Cache stampede fix
- ✓ Rate limiter
- ✓ Thread pool deadlock prevention

SCENARIOS

- ✓ Odd/Even printing
- ✓ N threads **in order**
- ✓ BlockingQueue **from** scratch
- ✓ Deadlock simulation + fix
- ✓ Stop thread gracefully
- ✓ Multiple producer single consumer
- ✓ Parallel task **with** timeout
- ✓ Lazy init patterns

MODERN JAVA

- ✓ Virtual threads (Java **21**)
- ✓ Structured concurrency (Java **21**)

DEBUGGING

- ✓ Thread dumps analysis
- ✓ Deadlock detection
- ✓ JFR, async-profiler
- ✓ Memory visibility bug detection

Nothing left uncovered — this is the complete Java Multithreading & Concurrency guide for 8 years experience level.

4/23/2026, 8:42:24 AM